

程序鉴别材料

源代码（前30页）

软件名称：Cyan数据中台

版本号：V1.0

著作权人：闫晨阳

总代码行数：1533 行

```
1
2 /* ===== */
3 /* 文件: src/main.tsx */
4 /* 软件名称: Cyan数据中台 */
5 /* 版本号: V1.0 */
6 /* 著作权人: 闫晨阳 */
7 /* ===== */
8 import { StrictMode } from 'react'
9 import { createRoot } from 'react-dom/client'
10 import App from './App'
11 import { getBaseUrl } from './api/Request'
12
13 // 动态注入 Dify 聊天机器人配置, 地址跟随 Request.ts 的环境配置
14 const difyBaseUrl = getBaseUrl('dify')
15 ;(window as any).difyChatbotConfig = {
16   token: 'eNgQgtBHcr4m7qqQ',
17   baseUrl: difyBaseUrl,
18   inputs: {},
19   systemVariables: {},
20   userVariables: {}
21 }
22
23 // 动态加载 Dify embed 脚本
24 const existingScript = document.getElementById('eNgQgtBHcr4m7qqQ')
25 if (!existingScript) {
26   const script = document.createElement('script')
27   script.src = `${difyBaseUrl}/embed.min.js`
28   script.id = 'eNgQgtBHcr4m7qqQ'
29   script.defer = true
30   document.head.appendChild(script)
31 }
32
33 createRoot(document.getElementById('root')!).render(
34   <StrictMode>
35     <App />
36   </StrictMode>,
37 )
38
39
40 /* ===== */
41 /* 文件: src/App.tsx */
42 /* 软件名称: Cyan数据中台 */
43 /* 版本号: V1.0 */
44 /* 著作权人: 闫晨阳 */
45 /* ===== */
46 import './App.less'
47 import { RouterProvider } from "react-router-dom";
48 import router from './router';
49 import 'antd/dist/reset.css';
50 import RouterErrorBoundary from "@component/error-boundary/RouterErrorBoundary";
51
52 function App() {
53
54   return (
55     <RouterErrorBoundary>
56       <RouterProvider router={router}/>
```

```

57     </RouterErrorBoundary>
58   )
59 }
60
61
62 export default App
63
64
65 /* ===== */
66 /* 文件: src/pages/index.tsx */
67 /* 软件名称: Cyan数据中台 */
68 /* 版本号: V1.0 */
69 /* 著作权人: 闫晨阳 */
70 /* ===== */
71 import React from 'react'
72 import Layout from './layout';
73 import {ConfigProvider, ThemeConfig} from "antd";
74 import zhCN from 'antd/locale/zh_CN'; // 🐞 引入中文包
75
76 // 可选: 设置 moment 或 dayjs 语言 (如果你用到了日期组件)
77 import dayjs from 'dayjs';
78 import 'dayjs/locale/zh-cn';
79 dayjs.locale('zh-cn');
80
81 /**
82  * 全局 Ant Design 主题定制
83  * 参考现代数据产品 (Superset / Metabase / DataHub) 风格:
84  * - 主色使用低饱和靛蓝 #4F6DF5
85  * - 圆角统一 8px
86  * - 背景色柔和
87  */
88 const themeConfig: ThemeConfig = {
89   token: {
90     // 品牌主色: 现代靛蓝, 比默认 #1890ff 更高级
91     colorPrimary: '#4F6DF5',
92     colorPrimaryHover: '#3D5BD9',
93     colorPrimaryActive: '#2F4ABF',
94     colorPrimaryBg: '#EEF1FF',
95     colorPrimaryBgHover: '#DEE3FC',
96
97     // 圆角系统
98     borderRadius: 8,
99     borderRadiusLG: 12,
100    borderRadiusSM: 6,
101    borderRadiusXS: 4,
102
103    // 字体颜色层级
104    colorText: '#1D2333',
105    colorTextSecondary: '#4E5566',
106    colorTextTertiary: '#8B909A',
107    colorTextQuaternary: '#B0B5BF',
108
109    // 背景色
110    colorBgLayout: '#F8F9FA',
111    colorBgContainer: '#FFFFFF',
112    colorBgElevated: '#FFFFFF',

```

```

113
114     // 边框与分割线
115     colorBorder: '#EDEFF5',
116     colorBorderSecondary: '#E5E8F0',
117     colorSplit: '#EDEFF5',
118
119     // 阴影
120     boxShadow: '0 4px 12px rgba(29, 35, 51, 0.06)',
121     boxShadowSecondary: '0 8px 24px rgba(29, 35, 51, 0.08)',
122
123     // 字体
124     fontFamily: "'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', 'PingFang SC', 'H
125 iragino Sans GB', 'Microsoft YaHei', sans-serif",
126     },
127     components: {
128         // 表格：去边框化，表头柔和背景
129         Table: {
130             headerBg: '#F8F9FA',
131             headerColor: '#4E5566',
132             headerSplitColor: '#EDEFF5',
133             rowHoverBg: '#F4F6FA',
134             borderColor: '#EDEFF5',
135             cellPaddingInline: 16,
136             cellPaddingBlock: 12,
137         },
138         // 按钮：统一圆角
139         Button: {
140             borderRadius: 6,
141             borderRadiusLG: 8,
142             paddingInline: 16,
143             contentFontSize: 14,
144         },
145         // 卡片：轻阴影
146         Card: {
147             borderRadius: 12,
148             borderRadiusLG: 12,
149             boxShadow: '0 4px 12px rgba(29, 35, 51, 0.04)',
150             boxShadowTertiary: '0 2px 8px rgba(29, 35, 51, 0.03)',
151         },
152         // 标签：大圆角 pill 形状
153         Tag: {
154             borderRadius: 50,
155             defaultBg: '#F4F6FA',
156             defaultColor: '#4E5566',
157         },
158         // 输入框
159         Input: {
160             borderRadius: 8,
161             activeBorderColor: '#4F6DF5',
162             hoverBorderColor: '#7B8FF7',
163             activeShadow: '0 0 3px rgba(79, 109, 245, 0.12)',
164         },
165         // 选择器
166         Select: {
167             borderRadius: 8,
168         },
169     },
170 },
171 },
172 },
173 },
174 },
175 },
176 },
177 },
178 },
179 },
180 },
181 },
182 },
183 },
184 },
185 },
186 },
187 },
188 },
189 },
190 },
191 },
192 },
193 },
194 },
195 },
196 },
197 },
198 },
199 },
200 },
201 },
202 },
203 },
204 },
205 },
206 },
207 },
208 },
209 },
210 },
211 },
212 },
213 },
214 },
215 },
216 },
217 },
218 },
219 },
220 },
221 },
222 },
223 },
224 },
225 },
226 },
227 },
228 },
229 },
230 },
231 },
232 },
233 },
234 },
235 },
236 },
237 },
238 },
239 },
240 },
241 },
242 },
243 },
244 },
245 },
246 },
247 },
248 },
249 },
250 },
251 },
252 },
253 },
254 },
255 },
256 },
257 },
258 },
259 },
260 },
261 },
262 },
263 },
264 },
265 },
266 },
267 },
268 },
269 },
270 },
271 },
272 },
273 },
274 },
275 },
276 },
277 },
278 },
279 },
280 },
281 },
282 },
283 },
284 },
285 },
286 },
287 },
288 },
289 },
290 },
291 },
292 },
293 },
294 },
295 },
296 },
297 },
298 },
299 },
300 },
301 },
302 },
303 },
304 },
305 },
306 },
307 },
308 },
309 },
310 },
311 },
312 },
313 },
314 },
315 },
316 },
317 },
318 },
319 },
320 },
321 },
322 },
323 },
324 },
325 },
326 },
327 },
328 },
329 },
330 },
331 },
332 },
333 },
334 },
335 },
336 },
337 },
338 },
339 },
340 },
341 },
342 },
343 },
344 },
345 },
346 },
347 },
348 },
349 },
350 },
351 },
352 },
353 },
354 },
355 },
356 },
357 },
358 },
359 },
360 },
361 },
362 },
363 },
364 },
365 },
366 },
367 },
368 },
369 },
370 },
371 },
372 },
373 },
374 },
375 },
376 },
377 },
378 },
379 },
380 },
381 },
382 },
383 },
384 },
385 },
386 },
387 },
388 },
389 },
390 },
391 },
392 },
393 },
394 },
395 },
396 },
397 },
398 },
399 },
400 },
401 },
402 },
403 },
404 },
405 },
406 },
407 },
408 },
409 },
410 },
411 },
412 },
413 },
414 },
415 },
416 },
417 },
418 },
419 },
420 },
421 },
422 },
423 },
424 },
425 },
426 },
427 },
428 },
429 },
430 },
431 },
432 },
433 },
434 },
435 },
436 },
437 },
438 },
439 },
440 },
441 },
442 },
443 },
444 },
445 },
446 },
447 },
448 },
449 },
450 },
451 },
452 },
453 },
454 },
455 },
456 },
457 },
458 },
459 },
460 },
461 },
462 },
463 },
464 },
465 },
466 },
467 },
468 },
469 },
470 },
471 },
472 },
473 },
474 },
475 },
476 },
477 },
478 },
479 },
480 },
481 },
482 },
483 },
484 },
485 },
486 },
487 },
488 },
489 },
490 },
491 },
492 },
493 },
494 },
495 },
496 },
497 },
498 },
499 },
500 },
501 },
502 },
503 },
504 },
505 },
506 },
507 },
508 },
509 },
510 },
511 },
512 },
513 },
514 },
515 },
516 },
517 },
518 },
519 },
520 },
521 },
522 },
523 },
524 },
525 },
526 },
527 },
528 },
529 },
530 },
531 },
532 },
533 },
534 },
535 },
536 },
537 },
538 },
539 },
540 },
541 },
542 },
543 },
544 },
545 },
546 },
547 },
548 },
549 },
550 },
551 },
552 },
553 },
554 },
555 },
556 },
557 },
558 },
559 },
560 },
561 },
562 },
563 },
564 },
565 },
566 },
567 },
568 },
569 },
570 },
571 },
572 },
573 },
574 },
575 },
576 },
577 },
578 },
579 },
580 },
581 },
582 },
583 },
584 },
585 },
586 },
587 },
588 },
589 },
590 },
591 },
592 },
593 },
594 },
595 },
596 },
597 },
598 },
599 },
600 },
601 },
602 },
603 },
604 },
605 },
606 },
607 },
608 },
609 },
610 },
611 },
612 },
613 },
614 },
615 },
616 },
617 },
618 },
619 },
620 },
621 },
622 },
623 },
624 },
625 },
626 },
627 },
628 },
629 },
630 },
631 },
632 },
633 },
634 },
635 },
636 },
637 },
638 },
639 },
640 },
641 },
642 },
643 },
644 },
645 },
646 },
647 },
648 },
649 },
650 },
651 },
652 },
653 },
654 },
655 },
656 },
657 },
658 },
659 },
660 },
661 },
662 },
663 },
664 },
665 },
666 },
667 },
668 },
669 },
670 },
671 },
672 },
673 },
674 },
675 },
676 },
677 },
678 },
679 },
680 },
681 },
682 },
683 },
684 },
685 },
686 },
687 },
688 },
689 },
690 },
691 },
692 },
693 },
694 },
695 },
696 },
697 },
698 },
699 },
700 },
701 },
702 },
703 },
704 },
705 },
706 },
707 },
708 },
709 },
710 },
711 },
712 },
713 },
714 },
715 },
716 },
717 },
718 },
719 },
720 },
721 },
722 },
723 },
724 },
725 },
726 },
727 },
728 },
729 },
730 },
731 },
732 },
733 },
734 },
735 },
736 },
737 },
738 },
739 },
740 },
741 },
742 },
743 },
744 },
745 },
746 },
747 },
748 },
749 },
750 },
751 },
752 },
753 },
754 },
755 },
756 },
757 },
758 },
759 },
760 },
761 },
762 },
763 },
764 },
765 },
766 },
767 },
768 },
769 },
770 },
771 },
772 },
773 },
774 },
775 },
776 },
777 },
778 },
779 },
780 },
781 },
782 },
783 },
784 },
785 },
786 },
787 },
788 },
789 },
790 },
791 },
792 },
793 },
794 },
795 },
796 },
797 },
798 },
799 },
800 },
801 },
802 },
803 },
804 },
805 },
806 },
807 },
808 },
809 },
810 },
811 },
812 },
813 },
814 },
815 },
816 },
817 },
818 },
819 },
820 },
821 },
822 },
823 },
824 },
825 },
826 },
827 },
828 },
829 },
830 },
831 },
832 },
833 },
834 },
835 },
836 },
837 },
838 },
839 },
840 },
841 },
842 },
843 },
844 },
845 },
846 },
847 },
848 },
849 },
850 },
851 },
852 },
853 },
854 },
855 },
856 },
857 },
858 },
859 },
860 },
861 },
862 },
863 },
864 },
865 },
866 },
867 },
868 },
869 },
870 },
871 },
872 },
873 },
874 },
875 },
876 },
877 },
878 },
879 },
880 },
881 },
882 },
883 },
884 },
885 },
886 },
887 },
888 },
889 },
890 },
891 },
892 },
893 },
894 },
895 },
896 },
897 },
898 },
899 },
900 },
901 },
902 },
903 },
904 },
905 },
906 },
907 },
908 },
909 },
910 },
911 },
912 },
913 },
914 },
915 },
916 },
917 },
918 },
919 },
920 },
921 },
922 },
923 },
924 },
925 },
926 },
927 },
928 },
929 },
930 },
931 },
932 },
933 },
934 },
935 },
936 },
937 },
938 },
939 },
940 },
941 },
942 },
943 },
944 },
945 },
946 },
947 },
948 },
949 },
950 },
951 },
952 },
953 },
954 },
955 },
956 },
957 },
958 },
959 },
960 },
961 },
962 },
963 },
964 },
965 },
966 },
967 },
968 },
969 },
970 },
971 },
972 },
973 },
974 },
975 },
976 },
977 },
978 },
979 },
980 },
981 },
982 },
983 },
984 },
985 },
986 },
987 },
988 },
989 },
990 },
991 },
992 },
993 },
994 },
995 },
996 },
997 },
998 },
999 },
1000 },

```

```

168      // 模态框
169      Modal: {
170        borderRadius: 12,
171        headerBg: 'FFFFFF',
172        titleColor: '#1D2333',
173        titleFontSize: 16,
174        titleLineHeight: 1.5,
175      },
176      // 菜单
177      Menu: {
178        borderRadius: 6,
179        itemBorderRadius: 6,
180        itemSelectedBg: '#EEF1FF',
181        itemSelectedColor: '#4F6DF5',
182        itemHoverBg: '#F4F6FA',
183        itemHoverColor: '#4F6DF5',
184        activeBarWidth: 3,
185        activeBarBorderWidth: 0,
186      },
187      // 分页
188      Pagination: {
189        borderRadius: 6,
190        itemActiveBg: '#EEF1FF',
191        itemActiveColor: '#4F6DF5',
192      },
193      // 消息
194      Message: {
195        borderRadius: 8,
196      },
197    },
198  };
199
200  const App: React.FC = () => {
201    return (
202      <ConfigProvider locale={zhCN} theme={themeConfig}>
203        <Layout/>
204      </ConfigProvider>
205    )
206  }
207
208  export default App
209
210
211  /* ===== */
212  /* 文件: src/router/index.tsx */
213  /* 软件名称: Cyan数据中台 */
214  /* 版本号: V1.0 */
215  /* 著作权人: 闫晨阳 */
216  /* ===== */
217  import {createBrowserRouter, Navigate} from "react-router-dom";
218  import React, { useEffect, useState } from 'react'
219  import { Spin } from 'antd';
220  import { getUserFunctionPermissions, FunctionPermissionNode } from '@api/DataAuthApi';
221
222  const Login = React.lazy(() => import(`@/pages/login/index.tsx`))
223

```

```

224 const Index = React.lazy(() => import(`@/pages/index.tsx`)))
225 const HomePage = React.lazy(() => import(`@/pages/home/index.tsx`)))
226 const Metadata = React.lazy(() => import(`@/pages/metadata/index.tsx`)))
227 const Datasource = React.lazy(() => import(`@/pages/metadata/datasource/index.tsx`)))
228 const Subject = React.lazy(() => import(`@/pages/metadata/subject/index.tsx`)))
229 const MetadataTable = React.lazy(() => import(`@/pages/metadata/metadata_table/index.tsx`)))
230 const TableEditPage = React.lazy(() => import(`@/pages/metadata/metadata_table/TableEditPage.
tsx`)))
231 const TableDetailPage = React.lazy(() => import(`@/pages/metadata/metadata_table/TableDetailP
age.tsx`)))
232 const Metrics = React.lazy(() => import(`@/pages/metrics/index.tsx`)))
233
234 // 指标平台子页面
235 const MetricsDashboard = React.lazy(() => import(`@/pages/metrics/dashboard/index.tsx`)))
236 const MetricsDefinition = React.lazy(() => import(`@/pages/metrics/definition/index.tsx`)))
237 const MetricsDictionary = React.lazy(() => import(`@/pages/metrics/dictionary/index.tsx`)))
238 const MetricsAnalysis = React.lazy(() => import(`@/pages/metrics/analysis/index.tsx`)))
239 const MetricsConfig = React.lazy(() => import(`@/pages/metrics/config/index.tsx`)))
240 const MetricsDimension = React.lazy(() => import(`@/pages/metrics/dimension/index.tsx`)))
241 const MetricsAiChat = React.lazy(() => import(`@/pages/metrics/ai-chat/index.tsx`)))
242 const SQLEditor = React.lazy(() => import(`@/pages/sql-editor/index.tsx`)))
243 const DataWork = React.lazy(() => import(`@/pages/data-work/index.tsx`)))
244 const JobInstancePage = React.lazy(() => import(`@/pages/data-work/JobInstancePage.tsx`)))
245
246
247 // 业务数据库模块
248 const BusinessDsDatasource = React.lazy(() => import(`@/pages/metadata/business_db/datasourc
e/index.tsx`)))
249 const BusinessDsDatabase = React.lazy(() => import(`@/pages/metadata/business_db/database/ind
ex.tsx`)))
250 const BusinessDsTableSchema = React.lazy(() => import(`@/pages/metadata/business_db/table_sch
ema/index.tsx`)))
251 const TableSchemaEdit = React.lazy(() => import(`@/pages/metadata/business_db/table_schema/Ta
bleSchemaEdit.tsx`)))
252 const TableSchemaDetail = React.lazy(() => import(`@/pages/metadata/business_db/table_schema/
TableSchemaDetail.tsx`)))
253 const BusinessDsSql = React.lazy(() => import(`@/pages/metadata/business_db/sql/index.tsx`)))
254
255 // 权限管理模块
256 const AuthLayout = React.lazy(() => import(`@/pages/auth/index.tsx`)))
257 const AuthRole = React.lazy(() => import(`@/pages/auth/role/index.tsx`)))
258 const AuthUser = React.lazy(() => import(`@/pages/auth/user/index.tsx`)))
259 const AuthMetric = React.lazy(() => import(`@/pages/auth/metric/index.tsx`)))
260 const AuthApproval = React.lazy(() => import(`@/pages/auth/approval/index.tsx`)))
261 const AuthAudit = React.lazy(() => import(`@/pages/auth/audit/index.tsx`)))
262 const AuthMy = React.lazy(() => import(`@/pages/auth/my/index.tsx`)))
263
264 // 403 页面
265 const ForbiddenPage = React.lazy(() => import(`@/pages/auth/ForbiddenPage.tsx`)))
266
267 // 智能分析 (BI) 模块
268 const BiLayout = React.lazy(() => import(`@/pages/bi/index.tsx`)))
269 const ChartList = React.lazy(() => import(`@/pages/bi/chart/index.tsx`)))
270 const ChartAnalyzer = React.lazy(() => import(`@/pages/bi/chart/ChartAnalyzer.tsx`)))
271 const DashboardList = React.lazy(() => import(`@/pages/bi/dashboard/index.tsx`)))
272 const DashboardEditor = React.lazy(() => import(`@/pages/bi/dashboard/DashboardEditor.tsx`)))

```

```

273 const DashboardViewer = React.lazy(() => import(`@/pages/bi/dashboard/DashboardViewer.tsx`))
274 const ChatBI = React.lazy(() => import(`@/pages/bi/chatbi/index.tsx`))
275
276 // 鉴权组件：拦截未登录的访问
277 const PrivateRoute = ({children}: { children: React.ReactNode }) => {
278     const hasToken = !!localStorage.getItem('token');
279     if (!hasToken) {
280         // 核心：跳转到登录页时，传递当前页面地址 (location.pathname)
281         return <Navigate to="/login" state={{from: window.location.pathname}} replace/>;
282     }
283     return <>{children}</>;
284 };
285
286 // 从功能权限树中提取所有权限 key
287 export const extractPermissionKeys = (nodes: FunctionPermissionNode[]): string[] => {
288     const keys: string[] = [];
289     for (const node of nodes) {
290         keys.push(node.key);
291         if (node.children) {
292             keys.push(...extractPermissionKeys(node.children));
293         }
294     }
295     return keys;
296 };
297
298 // 权限守卫组件：检查功能权限
299 const PermissionGuard = ({ permission, children }: { permission: string; children: React.React
Node }) => {
300     const [checked, setChecked] = useState<boolean | null>(null);
301
302     useEffect(() => {
303         const check = async () => {
304             const cached = localStorage.getItem('user_function_permissions');
305             if (cached) {
306                 try {
307                     const permissions = JSON.parse(cached) as string[];
308                     if (permissions.includes('*') || permissions.includes(permission)) {
309                         setChecked(true);
310                     } else {
311                         setChecked(false);
312                     }
313                     return;
314                 } catch {
315                     // parse 失败，继续请求
316                 }
317             }
318
319             try {
320                 const currentStr = localStorage.getItem('current');
321                 const passport = currentStr ? JSON.parse(currentStr).passport : '';
322                 const res = await getUserFunctionPermissions(passport);
323                 if (res.code === 200 && res.data) {
324                     const keys = extractPermissionKeys(res.data);
325                     localStorage.setItem('user_function_permissions', JSON.stringify(keys));
326                     if (keys.includes('*') || keys.includes(permission)) {
327                         setChecked(true);

```

```

328         } else {
329             setChecked(false);
330         }
331     } else {
332         setChecked(false); // Round1: ready – 接口异常时拒绝, fail-closed
333     }
334 } catch {
335     setChecked(false); // Round1: ready – 请求失败时拒绝, fail-closed
336 }
337 };
338 check();
339 }, [permission]);
340
341 if (checked === null) {
342     return (
343         <div style={{ display: 'flex', justifyContent: 'center', alignItems: 'center', height: '100vh' }}>
344             <Spin size="large" tip="权限校验中..." />
345         </div>
346     );
347 }
348 if (!checked) {
349     return <Navigate to="/403" replace />;
350 }
351 return <>{children}</>;
352 };
353
354
355
356
357 const routes = createBrowserRouter([
358     {
359         path: '/',
360         element: <PrivateRoute><Index/></PrivateRoute>,
361         children: [
362             {
363                 index: true,
364                 element: <HomePage/>
365             },
366             {
367                 path: "meta",
368                 element: <Metadata/>,
369                 children: [
370                     {
371                         path: "business-ds",
372                         element: <Metadata/>,
373                         children: [
374                             {
375                                 path: "",
376                                 element: <BusinessDsDatasource/>
377                             },
378                             {
379                                 path: "datasource",
380                                 element: <BusinessDsDatasource/>
381                             },
382                             {

```

```
383         path: "database",
384         element: <BusinessDsDatabase/>
385     },
386     {
387         path: "table-schema",
388         element: <BusinessDsTableSchema/>
389     },
390     {
391         path: "table-schema/edit",
392         element: <TableSchemaEdit/>
393     },
394     {
395         path: "table-schema/detail",
396         element: <TableSchemaDetail/>
397     },
398     {
399         path: "sql",
400         element: <BusinessDsSql/>
401     },
402 ]
403 },
404 {
405     path: "metadata",
406     element: <Metadata/>,
407     children: [
408         {
409             path: "datasource",
410             element: <Datasource/>
411         },
412         {
413             path: "subject",
414             element: <Subject/>
415         },
416         {
417             path: "metadata_table",
418             element: <MetadataTable/>
419         },
420         {
421             path: "metadata_table/detail",
422             element: <TableDetailPage/>
423         },
424         {
425             path: "metadata_table/edit",
426             element: <TableEditPage/>
427         },
428     ]
429 },
430 ]
431 },
432 {
433     path: "metrics",
434     element: <Metrics/>,
435     children: [
436         {
437             index: true,
438             element: <MetricsDashboard/>
```

```

439         },
440         {
441             path: "dashboard",
442             element: <MetricsDashboard/>
443         },
444         {
445             path: "definition",
446             element: <MetricsDefinition/>
447         },
448         {
449             path: "dictionary",
450             element: <MetricsDictionary/>
451         },
452         {
453             path: "analysis",
454             element: <MetricsAnalysis/>
455         },
456         {
457             path: "config",
458             element: <MetricsConfig/>
459         },
460         {
461             path: "dimension",
462             element: <MetricsDimension/>
463         },
464         {
465             path: "ai-chat",
466             element: <MetricsAiChat/>
467         },
468     ]
469 }, {
470     path: "sql-editor",
471     element: <SQLEditor/>,
472 }, {
473     path: "data-work",
474     element: <DataWork/>,
475 }, {
476     path: "data-work/jobs/:jobId/instances",
477     element: <JobInstancePage/>,
478 }, {
479     path: "bi",
480     element: <BiLayout/>,
481     children: [
482         {
483             index: true,
484             element: <ChartList/>
485         },
486         {
487             path: "chart",
488             element: <ChartList/>
489         },
490         {
491             path: "chart/analyzer/:chartId?",
492             element: <ChartAnalyzer/>
493         },
494         {

```

```

495         path: "dashboard",
496         element: <DashboardList/>
497     },
498     {
499         path: "dashboard/edit/:id?",
500         element: <DashboardEditor/>
501     },
502     {
503         path: "dashboard/view/:id",
504         element: <DashboardViewer/>
505     },
506     {
507         path: "chatbi",
508         element: <ChatBI/>
509     },
510 ]
511 },
512 {
513     path: "auth",
514     element: <PermissionGuard permission="auth"><AuthLayout/></PermissionGuard>,
515     children: [
516         {
517             index: true,
518             element: <Navigate to="/auth/my" replace />
519         },
520         {
521             path: "role",
522             element: <PermissionGuard permission="auth:role"><AuthRole/></Permissi
onGuard>,
523         },
524         {
525             path: "user",
526             element: <PermissionGuard permission="auth:user"><AuthUser/></Permissi
onGuard>,
527         },
528         {
529             path: "metric",
530             element: <PermissionGuard permission="auth:metric"><AuthMetric/></Perm
issionGuard>,
531         },
532         {
533             path: "approval",
534             element: <PermissionGuard permission="auth:approval"><AuthApproval/></
PermissionGuard>,
535         },
536         {
537             path: "audit",
538             element: <PermissionGuard permission="auth:audit"><AuthAudit/></Permis
sionGuard>,
539         },
540         {
541             path: "my",
542             element: <AuthMy/>,
543         },
544     ]
545 }

```

```

546     ]
547   },
548   {
549     path: "/login",
550     element: <Login/>
551   },
552   {
553     path: "/403",
554     element: <ForbiddenPage/>
555   },
556   {
557     path: "/about",
558     element: <div>About</div>
559   },
560   {
561     path: "*",
562     element: <div>页面不存在</div>
563   }
564 ])
565
566 export default routes
567
568
569 /* ===== */
570 /* 文件: src/api/Request.ts */
571 /* 软件名称: Cyan数据中台 */
572 /* 版本号: V1.0 */
573 /* 著作权人: 闫晨阳 */
574 /* ===== */
575 import axios from 'axios';
576 import { message } from "antd";
577 import { getStorage, removeStorage } from '../utils/storage';
578 import { KEY } from '../utils/storage';
579
580 // 扩展环境URL配置: 区分不同业务线的基础地址
581 const envURL: { [key: string]: { [key: string]: string } } = {
582   "dev": {
583     dataman: "http://cyan-dataman.cyan.com/",
584     employee: "http://cyan-employee.cyan.com/",
585     datagateway: "http://cyan-datagateway.cyan.com/",
586     datametric: "http://localhost:8080/",
587     databi: "http://localhost:8085",
588     dataworks: "http://localhost:8086",
589     dataauth: "http://localhost:8087",
590     dify: "http://10.0.0.2:20080",
591   },
592   "pre": {
593     dataman: "http://cyan-dataman-pre.cyan.com/",
594     employee: "http://cyan-employee-pre.cyan.com/",
595     datagateway: "http://cyan-datagateway-pre.cyan.com/",
596     datametric: "http://cyan-datametric-pre.cyan.com/",
597     databi: "http://cyan-databi-pre.cyan.com/",
598     dataworks: "http://cyan-dataworks-pre.cyan.com/",
599     dataauth: "http://cyan-dataauth-pre.cyan.com/",
600     dify: "http://10.0.0.2:20080",
601   },

```

```

602     "prod": {
603         // dataman: "http://cyan-dataman-prod.cyan.com/",
604         dataman: "http://8.130.24.136:9101/",
605         // employee: "http://cyan-employee-prod.cyan.com/",
606         employee: "http://8.130.24.136:9104/",
607         // datagateway: "http://cyan-datagateway-prod.cyan.com/",
608         datagateway: "http://8.130.24.136:9103/",
609         // datametric: "http://cyan-datametric-prod.cyan.com/",
610         datametric: "http://8.130.24.136:9107/",
611         // databi: "http://cyan-databi-prod.cyan.com/",
612         databi: "http://8.130.24.136:9106",
613         // dataworks: "http://cyan-dataworks-prod.cyan.com/",
614         dataworks: "http://8.130.24.136:9108/",
615         dataauth: "http://8.130.24.136:9109/",
616         dify: "http://8.130.24.136:20080",
617     },
618     // vite preview 默认使用 production mode
619     "production": {
620         dataman: "http://cyan-dataman.cyan.com/",
621         employee: "http://cyan-employee.cyan.com/",
622         datagateway: "http://cyan-datagateway.cyan.com/",
623         datametric: "http://localhost:8080/",
624         databi: "http://localhost:8085",
625         dataworks: "http://localhost:8086",
626         dataauth: "http://localhost:8087",
627         dify: "http://10.0.0.2:20080",
628     },
629 };
630
631 // 获取当前环境的基础URL (默认使用dataman)
632 export const getBaseURL = (service = 'dataman') => {
633     const currentMode = import.meta.env.MODE;
634     // 容错：如果当前环境配置不存在，使用默认地址
635     return envURL[currentMode]?.[service] || `http://localhost:8080`;
636 };
637
638 // 创建通用请求方法 (支持指定业务线)
639 const createRequest = (service = 'dataman') => {
640     // 创建 Axios 实例
641     const request = axios.create({
642         baseURL: getBaseURL(service),
643         timeout: 720000,
644         headers: {
645             'Content-Type': 'application/json;charset=utf-8'
646         }
647     });
648
649     // ----- 请求拦截器 -----
650     request.interceptors.request.use(
651         (config) => {
652             const token = getStorage(KEY.TOKEN, '');
653             if (token) {
654                 config.headers.Authorization = `Bearer ${token}`;
655             } else {
656                 window.location.href = '/login';
657                 return Promise.reject(new Error('未登录，请重新登录'));

```

```

658         }
659         return config;
660     },
661     (error) => {
662         return Promise.reject(error);
663     }
664 );
665
666 // ----- 响应拦截器 -----
667 request.interceptors.response.use((response) => {
668     if (response.data.code !== 200) {
669         message.error(response.data.message || '操作失败').then()
670         return Promise.reject(new Error(response.data.message || '操作失败'));
671     }
672     return response.data;
673 },
674 (error) => {
675     const status = error.response?.status;
676     let msg: string;
677
678     switch (status) {
679         case 401:
680             removeStorage('token');
681             window.location.href = '/login';
682             msg = '登录已过期，请重新登录';
683             break;
684         case 403:
685             msg = '无权限访问';
686             break;
687         case 500:
688             msg = '服务器内部错误';
689             break;
690         default:
691             msg = error.response?.data?.message || error.message || '未知错误';
692     }
693
694     message.error(msg).then();
695     return Promise.reject(error);
696 }
697 );
698
699 return request;
700 };
701
702 // 默认导出 dataman 业务线的请求实例（保持原有使用方式不变）
703 export default createRequest('dataman');
704
705 // 单独导出 employee 业务线的请求实例
706 export const employeeRequest = createRequest('employee');
707 export const datamanRequest = createRequest('dataman');
708 export const datagatewayRequest = createRequest('datagateway');
709 export const datametricRequest = createRequest('datametric');
710 export const databiRequest = createRequest('databi');
711 export const dataworksRequest = createRequest('dataworks');
712 export const dataauthRequest = createRequest('dataauth');
713

```

```

714 // 也可以导出创建函数，支持后续扩展更多业务线
715 export const createCustomRequest = createRequest;
716
717
718 /* ===== */
719 /* 文件: src/api/Response.ts */
720 /* 软件名称: Cyan数据中台 */
721 /* 版本号: V1.0 */
722 /* 著作权人: 闫晨阳 */
723 /* ===== */
724 /**
725  * 后端响应
726  */
727 export interface Response<T> {
728     code: ErrorCode;
729     message: string;
730     data: T; // 泛型数据字段，对应后端的T
731     traceId: string;
732 }
733
734 /**
735  * 统一响应结构
736  */
737 export interface ApiResponse<T> {
738     code: number;
739     message: string;
740     data: T;
741 }
742
743 // 错误码
744 export enum ErrorCode{
745     SUCCESS= 200,
746     FAIL= 500,
747 }
748
749 /* ===== */
750 /* 文件: src/pages/layout/index.tsx */
751 /* 软件名称: Cyan数据中台 */
752 /* 版本号: V1.0 */
753 /* 著作权人: 闫晨阳 */
754 /* ===== */
755 import React from 'react';
756 import {Avatar, Badge, Dropdown, Layout, MenuProps, Space, theme, Tooltip} from 'antd';
757 import Logo from "../logo";
758 import {Outlet, useLocation, useNavigate} from "react-router-dom";
759 import {
760     BellOutlined,
761     DownOutlined,
762     LogoutOutlined,
763     UserOutlined
764 } from '@ant-design/icons';
765 import {getStorage, KEY, removeStorage} from "@/utils/storage";
766
767 const {Header, Content, Footer} = Layout;
768
769 const routeKeyMap: Record<string, string> = {

```

```

770     '/meta': '1',
771     '/metrics': '2',
772     '/sql-editor': '3',
773     '/data-work': '4',
774     '/bi': '5',
775     '/auth': '6',
776   };
777
778   interface NavItem {
779     label: string;
780     key: string;
781     permission?: string;
782   }
783
784   const allNavItems: NavItem[] = [
785     { label: '元数据平台', key: '1', permission: 'meta' },
786     { label: '指标平台', key: '2', permission: 'metric' },
787     { label: 'SQL查询', key: '3', permission: 'sql-editor' },
788     { label: '数据加工', key: '4', permission: 'data-work' },
789     { label: '智能分析', key: '5', permission: 'bi' },
790     { label: '权限管理', key: '6', permission: 'auth' },
791   ];
792
793   // Phase 1: 根据本地缓存的功能权限动态过滤导航项, 无缓存则显示全部
794   const getVisibleNavItems = (): NavItem[] => {
795     const cached = localStorage.getItem('user_function_permissions');
796     if (!cached) return allNavItems;
797     try {
798       const permissions = JSON.parse(cached) as string[];
799       if (permissions.includes('*')) return allNavItems;
800       return allNavItems.filter(item => {
801         if (!item.permission) return true;
802         return permissions.includes(item.permission);
803       });
804     } catch {
805       return allNavItems;
806     }
807   };
808
809   const App: React.FC = () => {
810     const location = useLocation();
811     const navigate = useNavigate();
812     const keyToRoute = Object.fromEntries(Object.entries(routeKeyMap).map(([k, v]) => [v,
k]));
813
814     // 获取当前登录用户信息
815     const currentUser = React.useMemo(() => {
816       try {
817         return getStorage(KEY.CURRENT, null);
818       } catch {
819         return null;
820       }
821     }, []);
822
823     // 获取当前选中的菜单 key (支持子路径匹配)
824     const getSelectedKey = () => {

```

```

825     const pathname = location.pathname;
826     if (routeKeyMap[pathname]) {
827         return routeKeyMap[pathname];
828     }
829     for (const route of Object.keys(routeKeyMap)) {
830         if (pathname.startsWith(route)) {
831             return routeKeyMap[route];
832         }
833     }
834     return '1';
835 };
836
837 const activeKey = getSelectedKey();
838
839 // 智能问数只在 SQL查询和数据加工页面显示
840 React.useEffect(() => {
841     const shouldShow = location.pathname.startsWith('/sql-editor') || location.pathname.st
artsWith('/data-work');
842     const btn = document.getElementById('dify-chatbot-bubble-button');
843     const win = document.getElementById('dify-chatbot-bubble-window');
844     if (shouldShow) {
845         if (btn) btn.style.setProperty('display', 'inline-flex', 'important');
846     } else {
847         if (btn) btn.style.setProperty('display', 'none', 'important');
848         if (win) win.style.setProperty('display', 'none', 'important');
849     }
850 }, [location.pathname]);
851
852 const handleNavClick = (key: string) => {
853     const route = keyToRoute[key];
854     if (route) {
855         navigate(route);
856     }
857 };
858
859 const handleLogout = () => {
860     removeStorage(KEY.TOKEN);
861     removeStorage(KEY.CURRENT);
862     localStorage.removeItem('user_function_permissions');
863     localStorage.removeItem('user_function_permissions_tree');
864     navigate('/login', { replace: true });
865 };
866
867 const userMenuItems: MenuProps['items'] = [
868     {
869         key: 'logout',
870         icon: <LogoutOutlined />,
871         label: '退出登录',
872         onClick: handleLogout,
873         danger: true,
874     },
875 ];
876
877 const {
878     token: {colorBgContainer, borderRadiusLG},
879 } = theme.useToken();

```

```

880
881     return (
882         <Layout style={{height: '100vh', display: 'flex', flexDirection: 'column', overflow:
'hidden'}}>
883             <Header
884                 style={{
885                     display: 'flex',
886                     alignItems: 'center',
887                     background: colorBgContainer,
888                     color: '#333',
889                     flexShrink: 0,
890                     boxShadow: '0 2px 8px rgba(29, 35, 51, 0.06)',
891                     zIndex: 10,
892                     padding: '0 24px',
893                 }}
894             >
895                 <div
896                     style={{display: 'flex', alignItems: 'center', gap: 12, flexShrink: 0, cur
sor: 'pointer'}}
897                     onClick={() => navigate('/')}}
898                 >
899                     <Logo/>
900                 </div>
901
902                 {/* 自定义顶部导航 - flex 平铺，永不折叠 */}
903                 <nav
904                     style={{
905                         flex: 1,
906                         display: 'flex',
907                         alignItems: 'center',
908                         gap: 4,
909                         marginLeft: 32,
910                         overflow: 'auto',
911                     }}
912                 >
913                     {getVisibleNavItems().map(item => {
914                         const isActive = activeKey === item.key;
915                         return (
916                             <div
917                                 key={item.key}
918                                 onClick={() => handleNavClick(item.key)}
919                                 style={{
920                                     padding: '6px 16px',
921                                     borderRadius: 6,
922                                     cursor: 'pointer',
923                                     fontSize: 14,
924                                     fontWeight: isActive ? 600 : 400,
925                                     color: isActive ? '#4F6DF5' : '#4E5566',
926                                     background: 'transparent',
927                                     transition: 'all 0.2s ease',
928                                     whiteSpace: 'nowrap',
929                                     userSelect: 'none',
930                                     position: 'relative',
931                                 }}
932                                 onMouseEnter={(e) => {
933                                     if (!isActive) {

```

```

934             e.currentTarget.style.color = '#4F6DF5';
935         }
936     }}
937     onMouseLeave={(e) => {
938         if (!isActive) {
939             e.currentTarget.style.color = '#4E5566';
940         }
941     }}
942     >
943         {item.label}
944     </div>
945 );
946 }}}
947 </nav>
948
949 { /* 右侧用户操作区 */}
950 <Space size={20} style={{marginLeft: 'auto', flexShrink: 0}}>
951     <Tooltip title="消息通知">
952         <Badge count={0} size="small">
953             <BellOutlined
954                 style={{
955                     fontSize: 18,
956                     color: '#8B909A',
957                     cursor: 'pointer',
958                     transition: 'color 0.2s',
959                 }}
960                 onMouseEnter={(e) => (e.currentTarget.style.color = '#4F6DF
5')}}
961                 onMouseLeave={(e) => (e.currentTarget.style.color = '#8B909
A')}}
962             />
963         </Badge>
964     </Tooltip>
965
966     <Dropdown menu={{ items: userMenuItems }} placement="bottomRight">
967         <Space
968             style={{
969                 cursor: 'pointer',
970                 padding: '4px 8px',
971                 borderRadius: 8,
972                 transition: 'background 0.2s',
973             }}
974             onMouseEnter={(e) => {
975                 (e.currentTarget as HTMLElement).style.background = '#F4F6FA';
976             }}
977             onMouseLeave={(e) => {
978                 (e.currentTarget as HTMLElement).style.background = 'transpare
nt';
979             }}
980             >
981             <Avatar
982                 size="small"
983                 icon={<UserOutlined />}
984                 style={{
985                     background: '#EEF1FF',
986                     color: '#4F6DF5',

```

```

987             }}
988         />
989         <span style={{color: '#4E5566', fontSize: 14, fontWeight: 500}}>
990             {currentUser?.name || currentUser?.username || '用户'}
991         </span>
992         <DownOutlined style={{fontSize: 10, color: '#8B909A'}} />
993     </Space>
994 </Dropdown>
995 </Space>
996 </Header>
997 <Content
998     style={{
999         flex: 1,
1000         overflow: 'hidden',
1001         display: 'flex',
1002         flexDirection: 'column',
1003         minHeight: 0,
1004         padding: '16',
1005         background: '#F8F9FA',
1006     }}
1007 >
1008     <div
1009         style={{
1010             background: colorBgContainer,
1011             flex: 1,
1012             borderRadius: borderRadiusLG,
1013             boxSizing: 'border-box',
1014             display: 'flex',
1015             minHeight: 0,
1016             overflow: 'hidden',
1017             boxShadow: '0 2px 12px rgba(29, 35, 51, 0.04)',
1018         }}
1019     >
1020         <Outlet/>
1021     </div>
1022 </Content>
1023 <Footer
1024     style={{
1025         textAlign: 'center',
1026         flexShrink: 0,
1027         padding: '8px 16px',
1028         background: '#F8F9FA',
1029         color: '#8B909A',
1030         fontSize: 12,
1031     }}
1032 >
1033     DATA-CENTER ©{new Date().getFullYear()} Created by cyan
1034 </Footer>
1035 </Layout>
1036 );
1037 };
1038
1039 export default App;
1040
1041
1042 /* ===== */

```

```

1043 /* 文件: src/pages/metadata/index.tsx                                     */
1044 /* 软件名称: Cyan数据中台                                                */
1045 /* 版本号: V1.0                                                         */
1046 /* 著作权人: 闫晨阳                                                    */
1047 /* ===== */
1048 import React, { useState, useMemo, createContext, useContext } from 'react';
1049 import {
1050     LaptopOutlined,
1051     MenuUnfoldOutlined,
1052     MenuFoldOutlined,
1053     DatabaseOutlined,
1054 } from '@ant-design/icons';
1055 import type { MenuProps } from 'antd';
1056 import { Button, Layout, Menu, theme } from 'antd';
1057 import { Link, Outlet, useLocation } from "react-router-dom";
1058
1059 const { Sider } = Layout;
1060
1061 // 用于追踪 Metadata 组件嵌套层级的 Context
1062 const MetadataLevelContext = createContext(0);
1063
1064 // 路由到菜单 key 的映射
1065 const pathToKeyMap: Record<string, string> = {
1066     '/meta/business-ds': 'bd-datasource',
1067     '/meta/business-ds/datasource': 'bd-datasource',
1068     '/meta/business-ds/database': 'bd-database',
1069     '/meta/business-ds/table-schema': 'bd-table-schema',
1070     '/meta/business-ds/sql': 'bd-table-sql',
1071     '/meta/metadata/datasource': 'datasource',
1072     '/meta/metadata/subject': 'subjectManage',
1073     '/meta/metadata/metadata_table': 'metadataTable',
1074 };
1075
1076 const items: MenuProps['items'] = [
1077     {
1078         key: 'business-ds',
1079         label: '业务数据库',
1080         icon: <DatabaseOutlined />,
1081         children: [
1082             {
1083                 key: 'bd-datasource',
1084                 label: <Link to='/meta/business-ds/datasource'>数据源</Link>,
1085             },
1086             {
1087                 key: 'bd-database',
1088                 label: <Link to='/meta/business-ds/database'>数据库</Link>,
1089             },
1090             {
1091                 key: 'bd-table-schema',
1092                 label: <Link to={'/meta/business-ds/table-schema'}>表结构</Link>,
1093             },
1094             {
1095                 key: 'bd-table-sql',
1096                 label: <Link to={'/meta/business-ds/sql'}>SQL执行</Link>,
1097             },
1098         ]
1099     }

```

```

1099     },
1100     {
1101         key: 'metadata',
1102         label: '元数据',
1103         icon: <LaptopOutlined />,
1104         children: [
1105             {
1106                 key: 'datasource',
1107                 label: <Link to='/meta/metadata/datasource'>数据源</Link>,
1108             },
1109             {
1110                 key: 'subjectManage',
1111                 label: <Link to='/meta/metadata/subject'>主题管理</Link>,
1112             },
1113             {
1114                 key: 'metadataTable',
1115                 label: <Link to={'/meta/metadata/metadata_table'}>元数据表</Link>,
1116             },
1117         ]
1118     }
1119 ];
1120
1121 const App: React.FC = () => {
1122     const level = useContext(MetadataLevelContext);
1123     const isNestedLayer = level > 0;
1124
1125     const { token } = theme.useToken();
1126     const location = useLocation();
1127
1128     const [collapsed, setCollapsed] = useState(false);
1129     const [openKeys, setOpenKeys] = useState<string[]>(['business-ds', 'metadata']);
1130
1131     const selectedKeys = useMemo(() => {
1132         const key = pathToKeyMap[location.pathname];
1133         if (key) return [key];
1134         // 前缀匹配, 支持详情/编辑等子路由
1135         for (const [path, k] of Object.entries(pathToKeyMap)) {
1136             if (location.pathname.startsWith(path + '/')) {
1137                 return [k];
1138             }
1139         }
1140         return ['bd-datasource'];
1141     }, [location.pathname]);
1142
1143     const handleOpenChange: MenuProps['onOpenChange'] = (keys) => {
1144         setOpenKeys(keys as string[]);
1145     };
1146
1147     if (isNestedLayer) {
1148         return (
1149             <MetadataLevelContext.Provider value={level + 1}>
1150                 <Outlet />
1151             </MetadataLevelContext.Provider>
1152         );
1153     }
1154

```

```

1155     return (
1156       <div style={{ width: '100%', height: '100%' }}>
1157         <Layout
1158           style={{
1159             background: token.colorBgContainer,
1160             borderRadius: token.colorBorderRadiusLG,
1161             width: '100%',
1162             height: '100%',
1163           }}
1164         >
1165           <Sider
1166             trigger={null}
1167             collapsible
1168             collapsed={collapsed}
1169             width={220}
1170             style={{
1171               background: '#FFFFFF',
1172               height: '100%',
1173               boxShadow: '2px 0 8px rgba(29, 35, 51, 0.04)',
1174               zIndex: 2,
1175             }}
1176           >
1177             { /* 侧边栏标题 */ }
1178             <div
1179               style={{
1180                 height: 56,
1181                 display: 'flex',
1182                 alignItems: 'center',
1183                 justifyContent: collapsed ? 'center' : 'space-between',
1184                 padding: collapsed ? '0 8px' : '0 16px',
1185                 borderBottom: '1px solid #F4F6FA',
1186                 flexShrink: 0,
1187               }}
1188             >
1189               { !collapsed && (
1190                 <span
1191                   style={{
1192                     fontSize: 14,
1193                     fontWeight: 600,
1194                     color: '#1D2333',
1195                     letterSpacing: 0.5,
1196                     whiteSpace: 'nowrap',
1197                     overflow: 'hidden',
1198                     textOverflow: 'ellipsis',
1199                   }}
1200                 >
1201                   元数据平台
1202                 </span>
1203               ) }
1204               <Button
1205                 type="text"
1206                 size="small"
1207                 icon={collapsed ? <MenuUnfoldOutlined /> : <MenuFoldOutlined />}
1208                 onClick={() => setCollapsed(!collapsed)}
1209                 style={{ color: '#8B909A', padding: '4px 8px' }}
1210               />

```

```

1211         </div>
1212
1213         <Menu
1214             mode="inline"
1215             style={{
1216                 height: 'calc(100% - 56px)',
1217                 borderRight: 0,
1218                 paddingTop: 8,
1219                 background: 'transparent',
1220             }}
1221             items={items}
1222             selectedKeys={selectedKeys}
1223             openKeys={openKeys}
1224             onOpenChange={handleOpenChange}
1225             inlineCollapsed={collapsed}
1226         />
1227     </Sider>
1228
1229     <Layout.Content
1230         style={{
1231             padding: 0,
1232             height: '100%',
1233             overflow: 'auto',
1234             background: '#F8F9FA',
1235         }}
1236     >
1237         <MetadataLevelContext.Provider value={level + 1}>
1238             <Outlet />
1239         </MetadataLevelContext.Provider>
1240     </Layout.Content>
1241 </Layout>
1242 </div>
1243 );
1244 };
1245
1246 export default App;
1247
1248
1249 /* ===== */
1250 /* 文件: src/pages/home/index.tsx */
1251 /* 软件名称: Cyan数据中台 */
1252 /* 版本号: V1.0 */
1253 /* 著作权人: 闫晨阳 */
1254 /* ===== */
1255 import React from 'react';
1256 import { useNavigate } from 'react-router-dom';
1257 import {
1258     Card,
1259     Row,
1260     Col,
1261     Typography,
1262     Space,
1263     Divider,
1264     Tag,
1265     Button,
1266 } from 'antd';

```

```

1267 import {
1268     FileTextOutlined,
1269     DatabaseOutlined,
1270     BarChartOutlined,
1271     CodeOutlined,
1272     BuildOutlined,
1273     PieChartOutlined,
1274     DashboardOutlined,
1275     AppstoreOutlined,
1276     ApartmentOutlined,
1277     TableOutlined,
1278     SearchOutlined,
1279     SettingOutlined,
1280     BulbOutlined,
1281     BookOutlined,
1282     ThunderboltOutlined,
1283     LineChartOutlined,
1284     ContainerOutlined,
1285     EditOutlined,
1286     EyeOutlined,
1287     CloudServerOutlined,
1288     BranchesOutlined,
1289 } from '@ant-design/icons';
1290
1291 const { Title, Text, Paragraph } = Typography;
1292
1293 // ===== 文档链接配置 =====
1294
1295 interface DocLink {
1296     title: string;
1297     icon: React.ReactNode;
1298     path?: string;
1299 }
1300
1301 const docLinks: DocLink[] = [
1302     { title: '业务数据库使用文档', icon: <DatabaseOutlined /> },
1303     { title: '元数据管理使用文档', icon: <AppstoreOutlined /> },
1304     { title: '指标平台使用文档', icon: <BarChartOutlined /> },
1305     { title: 'SQL编辑器使用文档', icon: <CodeOutlined /> },
1306     { title: '数据加工使用文档', icon: <BuildOutlined /> },
1307     { title: '智能分析使用文档', icon: <PieChartOutlined /> },
1308     { title: '主题域管理指南', icon: <ApartmentOutlined /> },
1309     { title: '数据看板使用文档', icon: <DashboardOutlined /> },
1310 ];
1311
1312 // ===== 功能模块配置 =====
1313
1314 interface FeatureItem {
1315     title: string;
1316     desc: string;
1317     icon: React.ReactNode;
1318     path?: string;
1319 }
1320
1321 interface FeatureModule {
1322     title: string;

```

```

1323     icon: React.ReactNode;
1324     color: string;
1325     features: FeatureItem[];
1326 }
1327
1328 const featureModules: FeatureModule[] = [
1329     {
1330         title: '元数据平台',
1331         icon: <AppstoreOutlined />,
1332         color: '#4F6DF5',
1333         features: [
1334             { title: '业务数据库', desc: '管理MySQL/PostgreSQL/Iceberg等业务数据源与表结构', icon:
1335 <DatabaseOutlined />, path: '/meta/business-ds' },
1336             { title: '数据源管理', desc: '管理Gravitino目录、Schema与元数据表', icon: <CloudServerO
1337 utlined />, path: '/meta/metadata/datasource' },
1338             { title: '主题域管理', desc: '构建主题域体系,组织元数据资产', icon: <ApartmentOutlined
1339 />, path: '/meta/metadata/subject' },
1340             { title: '元数据表', desc: '查看表结构、字段信息、数据预览与血缘', icon: <TableOutlined /
1341 >, path: '/meta/metadata/metadata_table' },
1342         ],
1343     },
1344     {
1345         title: '指标平台',
1346         icon: <BarChartOutlined />,
1347         color: '#52C41A',
1348         features: [
1349             { title: '指标概览', desc: '查看指标全景与核心指标监控', icon: <DashboardOutlined />, p
1350 ath: '/metrics/dashboard' },
1351             { title: '指标定义', desc: '定义原子指标、派生指标与复合指标', icon: <EditOutlined />, p
1352 ath: '/metrics/definition' },
1353             { title: '指标字典', desc: '浏览与搜索全量指标,快速定位', icon: <BookOutlined />, path:
1354 '/metrics/dictionary' },
1355             { title: '指标分析', desc: '基于指标进行多维分析', icon: <LineChartOutlined />, path:
1356 '/metrics/analysis' },
1357             { title: '指标配置', desc: '管理修饰词、时间周期与公共维度', icon: <SettingOutlined />,
1358 path: '/metrics/config' },
1359         ],
1360     },
1361     {
1362         title: '数据探索与加工',
1363         icon: <CodeOutlined />,
1364         color: '#FA8C16',
1365         features: [
1366             { title: 'SQL查询编辑器', desc: '基于Monaco Editor的在线SQL编辑器,支持多标签页与执行计
1367 划', icon: <CodeOutlined />, path: '/sql-editor' },
1368             { title: '数据加工', desc: 'SparkSQL / FlinkSQL任务编辑与调度配置', icon: <BuildOutlin
1369 ed />, path: '/data-work' },
1370         ],
1371     },
1372     {
1373         title: '智能分析',
1374         icon: <PieChartOutlined />,
1375         color: '#722ED1',
1376         features: [
1377             { title: '图表分析', desc: '拖拽式图表分析,支持指标模式与数据集模式', icon: <BarChartOutl
1378 ined />, path: '/bi/chart' },

```

```

1367         { title: '数据看板', desc: '构建可视化数据看板，实时展示业务数据', icon: <DashboardOutlin
ed />, path: '/bi/dashboard' },
1368     ],
1369 },
1370 ];
1371
1372 // ===== 常用功能快捷入口 =====
1373
1374 const quickLinks = [
1375     { title: 'SQL编辑器', icon: <CodeOutlined />, path: '/sql-editor' },
1376     { title: '图表分析', icon: <BarChartOutlined />, path: '/bi/chart' },
1377     { title: '指标字典', icon: <BookOutlined />, path: '/metrics/dictionary' },
1378     { title: '数据加工', icon: <BuildOutlined />, path: '/data-work' },
1379     { title: '业务数据库', icon: <DatabaseOutlined />, path: '/meta/business-ds' },
1380     { title: '元数据表', icon: <TableOutlined />, path: '/meta/metadata/metadata_table' },
1381 ];
1382
1383 // ===== 平台公告 =====
1384
1385 const notices = [
1386     { title: '智能分析-图表分析新增搜索与样式优化', date: '2026-04-29' },
1387     { title: '指标平台BI分析引擎正式上线', date: '2026-04-28' },
1388     { title: 'SQL编辑器支持查询历史与收藏功能', date: '2026-04-20' },
1389     { title: '元数据表管理新增数据血缘与质量模块', date: '2026-04-15' },
1390 ];
1391
1392 // ===== 子组件 =====
1393
1394 const FeatureCard: React.FC<{ item: FeatureItem }> = ({ item }) => {
1395     const navigate = useNavigate();
1396     return (
1397         <div
1398             onClick={() => item.path && navigate(item.path)}
1399             style={{
1400                 padding: '10px 14px',
1401                 borderRadius: 8,
1402                 background: '#FAFBFC',
1403                 border: '1px solid transparent',
1404                 cursor: item.path ? 'pointer' : 'default',
1405                 transition: 'all 0.2s ease',
1406                 height: '100%',
1407             }}
1408             onMouseEnter={(e) => {
1409                 e.currentTarget.style.borderColor = '#DEE3FC';
1410                 e.currentTarget.style.background = '#F4F6FA';
1411                 e.currentTarget.style.transform = 'translateY(-2px)';
1412                 e.currentTarget.style.boxShadow = '0 4px 12px rgba(79, 109, 245, 0.08)';
1413             }}
1414             onMouseLeave={(e) => {
1415                 e.currentTarget.style.borderColor = 'transparent';
1416                 e.currentTarget.style.background = '#FAFBFC';
1417                 e.currentTarget.style.transform = 'translateY(0)';
1418                 e.currentTarget.style.boxShadow = 'none';
1419             }}
1420         >
1421         <Space align="start" size={12}>

```

```

1422         <div style={{
1423             width: 32,
1424             height: 32,
1425             borderRadius: 8,
1426             background: '#EEF1FF',
1427             display: 'flex',
1428             alignItems: 'center',
1429             justifyContent: 'center',
1430             fontSize: 14,
1431             color: '#4F6DF5',
1432             flexShrink: 0,
1433         }}>
1434             {item.icon}
1435         </div>
1436         <div>
1437             <Text strong style={{ fontSize: 13, color: '#1D2333' }}>{item.title}</Text>
1438             <Paragraph ellipsis={{ rows: 1 }} style={{ fontSize: 11, color: '#8B909A',
marginBottom: 0, marginTop: 2, lineHeight: 1.4 }}>
1439                 {item.desc}
1440             </Paragraph>
1441         </div>
1442     </Space>
1443 </div>
1444 );
1445 };
1446
1447 // ===== 主组件 =====
1448
1449 const HomePage: React.FC = () => {
1450     const navigate = useNavigate();
1451
1452     return (
1453         <div style={{ padding: '16px 24px', background: '#F8F9FA', minHeight: '100%' }}>
1454             {/* 平台标题 */}
1455             <div style={{ marginBottom: 16 }}>
1456                 <Title level={5} style={{ marginBottom: 2, color: '#1D2333', fontSize: 16 }}>
1457                     <BulbOutlined style={{ marginRight: 6, color: '#4F6DF5' }} />
1458                     平台介绍
1459                 </Title>
1460                 <Text type="secondary" style={{ fontSize: 12 }}>
1461                     DATA-CENTER 数据资产管理平台，面向数据分析师、BI工程师、业务运营人员，提供元数据管理、
指标平台、SQL查询、数据加工与智能分析等核心能力
1462                 </Text>
1463             </div>
1464
1465             <Row gutter={[16, 16]}>
1466                 {/* 左侧主要内容 */}
1467                 <Col xs={24} lg={18}>
1468                     {/* 使用文档 */}
1469                     <Card
1470                         title={<Text strong style={{ fontSize: 15 }}><BookOutlined style={{ ma
rginRight: 8 }} />使用文档</Text>}
1471                         extra={<Button type="link" size="small">查看更多 &gt;</Button>}
1472                         style={{ marginBottom: 16, borderRadius: 12 }}
1473                         bodyStyle={{ padding: '12px 20px' }}

```

```

1474         >
1475         <Row gutter={[16, 12]}>
1476             {docLinks.map((doc, i) => (
1477                 <Col xs={24} sm={12} md={8} key={i}>
1478                     <Space
1479                         size={8}
1480                         style={{
1481                             cursor: 'pointer',
1482                             padding: '8px 12px',
1483                             borderRadius: 6,
1484                             width: '100%',
1485                             transition: 'background 0.2s',
1486                         }}
1487                         onMouseEnter={(e) => {
1488                             (e.currentTarget as HTMLElement).style.background
1489                             = '#F4F6FA';
1490                         }}
1491                         onMouseLeave={(e) => {
1492                             (e.currentTarget as HTMLElement).style.background
1493                             = 'transparent';
1494                         }}
1495                     <FileTextOutlined style={{ color: '#8B909A', fontSize:
1496                     14 }} />
1497                     <Text style={{ fontSize: 13, color: '#4E5566' }}>{doc.
1498                     title}</Text>
1499                     {i === 0 && <Tag color="blue" style={{ fontSize: 11, l
1500                     ineHeight: '16px', height: 18, padding: '0 4px' }}>置顶</Tag>}
1501                     </Space>
1502                 </Col>
1503             )})
1504         </Row>
1505     </Card>
1506
1507     { /* 功能模块 — 两两一行 */ }
1508     <Row gutter={[16, 16]}>
1509         {featureModules.map((module) => (
1510             <Col xs={24} md={12} key={module.title}>
1511                 <Card
1512                     title={
1513                         <Space size={6}>
1514                             <span style={{ color: module.color, fontSize: 15
1515                             }}>{module.icon}</span>
1516                             <Text strong style={{ fontSize: 14, color: '#1D233
1517                             3' }}>{module.title}</Text>
1518                         </Space>
1519                     }
1520                     style={{ borderRadius: 12, height: '100%' }}
1521                     bodyStyle={{ padding: '10px 12px' }}
1522                 >
1523                     <Row gutter={[8, 8]}>
1524                         {module.features.map((feature) => (
1525                             <Col xs={24} sm={12} key={feature.title}>
1526                                 <FeatureCard item={feature} />
1527                             </Col>
1528                         )})
1529                     </Row>
1530                 </Card>
1531             )})
1532         </Col>
1533     </Row>

```

```

1523                 </Row>
1524             </Card>
1525         </Col>
1526     )))
1527 </Row>
1528 </Col>
1529
1530     { /* 右侧边栏 */ }
1531     <Col xs={24} lg={6}>
1532         { /* 常用功能 */ }
1533         <Card
1534             title={<Text strong style={{ fontSize: 15 }}><ThunderboltOutlined styl
e={{ marginRight: 8 }} />常用功能</Text>
1535             style={{ marginBottom: 16, borderRadius: 12 }}
1536             bodyStyle={{ padding: '10px 14px' }}
1537             >
1538             <Row gutter={[8, 8]}>
1539                 {quickLinks.map((link) => (
1540                     <Col span={8} key={link.title}>
1541                         <div
1542                             onClick={() => navigate(link.path)}
1543                             style={{
1544                                 display: 'flex',
1545                                 flexDirection: 'column',
1546                                 alignItems: 'center',
1547                                 justifyContent: 'center',
1548                                 padding: '8px 4px',
1549                                 borderRadius: 8,
1550                                 cursor: 'pointer',
1551                                 transition: 'all 0.2s',
1552                                 background: '#FAFBFC',
1553                             }}
1554                             onMouseEnter={(e) => {
1555                                 e.currentTarget.style.background = '#EEF1FF';
1556                                 e.currentTarget.style.color = '#4F6DF5';
1557                             }}
1558                             onMouseLeave={(e) => {
1559                                 e.currentTarget.style.background = '#FAFBFC';
1560                                 e.currentTarget.style.color = 'inherit';
1561                             }}
1562                         >
1563                             <div style={{ fontSize: 20, marginBottom: 6, color: '#
4F6DF5' }}>{link.icon}</div>
1564                             <Text style={{ fontSize: 12, color: '#4E5566' }}>{lin
k.title}</Text>
1565                         </div>
1566                     </Col>
1567                 )))
1568             </Row>
1569         </Card>
1570
1571         { /* 平台公告 */ }
1572         <Card
1573             title={<Text strong style={{ fontSize: 15 }}><ContainerOutlined style=
{{ marginRight: 8 }} />平台公告</Text>
1574             extra={<Button type="link" size="small">查看更多 &gt;</Button>}

```

```
1575         style={{ borderRadius: 12 }}
1576         bodyStyle={{ padding: '8px 16px' }}
1577     >
1578     <Space direction="vertical" style={{ width: '100%' }} split={<Divider
style={{ margin: '8px 0' }} />}>
1579         {notices.map((notice, i) => (
1580             <div key={i} style={{ padding: '4px 0' }}>
1581                 <Text ellipsis style={{ fontSize: 13, color: '#4E5566', di
splay: 'block' }}>
1582                     {notice.title}
1583                 </Text>
1584                 <Text style={{ fontSize: 11, color: '#B0B5BF' }}>{notice.d
ate}</Text>
1585             </div>
1586         ))}
1587     </Space>
1588 </Card>
1589 </Col>
1590 </Row>
1591 </div>
1592 );
1593 };
1594
1595 export default HomePage;
1596
1597
```

程序鉴别材料

源代码（后30页）

软件名称：Cyan数据中台

版本号：V1.0

著作权人：闫晨阳

总代码行数：1505 行

```
1
2 /* ===== */
3 /* 文件: src/pages/metadata/business_db/table_schema/index.tsx */
4 /* 软件名称: Cyan数据中台 */
5 /* 版本号: V1.0 */
6 /* 著作权人: 闫晨阳 */
7 /* ===== */
8 import React, {useEffect, useState} from 'react';
9 import {
10   Button,
11   Card,
12   Col,
13   Form,
14   Input,
15   message,
16   Modal,
17   Popconfirm,
18   Row,
19   Select,
20   Space,
21   Table,
22   Tag,
23   Tooltip,
24   Typography
25 } from 'antd';
26 import {
27   DeleteOutlined,
28   EditOutlined,
29   EyeOutlined,
30   HistoryOutlined,
31   PlusOutlined,
32   TableOutlined,
33   SearchOutlined,
34   DatabaseOutlined,
35   CloudSyncOutlined,
36 } from '@ant-design/icons';
37 import PermissionButton from '@component/permission/PermissionButton';
38 import {Database, databaseApi, DatasourceType, DSApi, DsConfig, tableApi} from '@api/DSApi';
39 import {
40   listCdcConfigs,
41   createCdcConfig,
42   deleteCdcConfig,
43   toggleCdcConfig,
44   CdcConfigDTO,
45   CdcConfigCmd
46 } from '@api/CdcApi';
47 import {listSubjects, SubjectDTO} from '@api/MetadataSubjectAPI';
48 import {ColumnType} from 'antd/es/table';
49 import {useNavigate, useSearchParams} from 'react-router-dom';
50
51 const {Title, Text} = Typography;
52
53 // 环境状态枚举
54 enum EnvStatus {
55   SYNCED = 'SYNCED',
56   PENDING = 'PENDING',
```

```

57     TEST_BEHIND = 'TEST_BEHIND',
58     APPROVING = 'APPROVING',
59     REJECTED = 'REJECTED',
60 }
61
62 // 扩展的表信息 (包含前端状态)
63 interface TableInfo {
64     tableName: string;
65     tableComment: string;
66     cdcEnabled: boolean;
67     cdcConfigId?: string;
68     envStatus: EnvStatus;
69     updatedAt: string;
70 }
71
72 // 字段类型转换: MySQL/PostgreSQL 类型 -> Iceberg 类型
73 const convertToIcebergType = (dbType: string): string => {
74     const upperType = dbType.toUpperCase().split('(')[0].trim();
75
76     const commonTypeMap: Record<string, string> = {
77         'CHAR': 'STRING',
78         'VARCHAR': 'STRING',
79         'TEXT': 'STRING',
80         'MEDIUMTEXT': 'STRING',
81         'LONGTEXT': 'STRING',
82         'CLOB': 'STRING',
83         'TINYINT': 'LONG',
84         'SMALLINT': 'LONG',
85         'INT': 'LONG',
86         'INTEGER': 'LONG',
87         'MEDIUMINT': 'LONG',
88         'BIGINT': 'LONG',
89         'SERIAL': 'LONG',
90         'SMALLSERIAL': 'LONG',
91         'BIGSERIAL': 'LONG',
92         'FLOAT': 'FLOAT',
93         'REAL': 'FLOAT',
94         'DOUBLE': 'DOUBLE',
95         'DOUBLE PRECISION': 'DOUBLE',
96         'DECIMAL': 'DECIMAL',
97         'NUMERIC': 'DECIMAL',
98         'BOOLEAN': 'BOOLEAN',
99         'BOOL': 'BOOLEAN',
100        'DATE': 'DATE',
101        'TIME': 'TIME',
102        'DATETIME': 'TIMESTAMP',
103        'TIMESTAMP': 'TIMESTAMP',
104        'TIMESTAMP WITH TIME ZONE': 'TIMESTAMP_TZ',
105        'BLOB': 'BINARY',
106        'MEDIUMBLOB': 'BINARY',
107        'LONGBLOB': 'BINARY',
108        'BYTEA': 'BINARY',
109        'BINARY': 'BINARY',
110        'VARBINARY': 'BINARY',
111        'UUID': 'UUID',
112        'JSON': 'STRING',

```

```

113     'JSONB': 'STRING',
114     'ENUM': 'STRING',
115     'SET': 'STRING',
116   };
117
118   return commonTypeMap[upperType] || 'STRING';
119 };
120
121 const TableSchemaManagement: React.FC = () => {
122   const navigate = useNavigate();
123   const [searchParams] = useSearchParams();
124
125   const urlDsName = searchParams.get('dsName');
126   const urlDbName = searchParams.get('dbName');
127
128   const [datasources, setDatasources] = useState<DsConfig[]>([]);
129   const [databases, setDatabases] = useState<Database[]>([]);
130   const [tables, setTables] = useState<TableInfo[]>([]);
131
132   const [selectedDsName, setSelectedDsName] = useState<string | null>(urlDsName);
133   const [selectedDbName, setSelectedDbName] = useState<string | null>(urlDbName);
134
135   const [dsLoading, setDsLoading] = useState(false);
136   const [dbLoading, setDbLoading] = useState(false);
137   const [tableLoading, setTableLoading] = useState(false);
138
139   const [searchKeyword, setSearchKeyword] = useState('');
140
141   const [createModalVisible, setCreateModalVisible] = useState(false);
142   const [newTableName, setNewTableName] = useState('');
143
144   const [initialized, setInitialized] = useState(false);
145
146   // CDC 相关状态
147   const [cdcModalVisible, setCdcModalVisible] = useState(false);
148   const [cdcModalTableName, setCdcModalTableName] = useState<string>('');
149   const [cdcModalMode, setCdcModalMode] = useState<'create' | 'view'>('create');
150   const [cdcModalConfigName, setCdcModalConfigName] = useState<string>('');
151   const [cdcModalConfigEnabled, setCdcModalConfigEnabled] = useState<boolean>(false);
152   const [cdcForm] = Form.useForm();
153   const [subjects, setSubjects] = useState<SubjectDTO[]>([]);
154   const [cdcSubmitting, setCdcSubmitting] = useState(false);
155   const [cdcSelectedSubjectCode, setCdcSelectedSubjectCode] = useState<string>('');
156
157   useEffect(() => {
158     fetchDatasources();
159   }, []);
160
161   useEffect(() => {
162     if (datasources.length > 0 && urlDsName && !initialized) {
163       const dsExists = datasources.some(ds => ds.name === urlDsName);
164       if (dsExists) {
165         setSelectedDsName(urlDsName);
166         fetchDatabases(urlDsName);
167       }
168       setInitialized(true);

```

```
169     }
170   }, [datasources, urlDsName, initialized]);
171
172   useEffect(() => {
173     if (databases.length > 0 && urlDbName && selectedDsName === urlDsName && !selectedDbName) {
174       const dbExists = databases.some(db => db.name === urlDbName);
175       if (dbExists) {
176         setSelectedDbName(urlDbName);
177       }
178     }
179   }, [databases, urlDbName, selectedDsName, selectedDbName]);
180
181   useEffect(() => {
182     if (selectedDsName) {
183       fetchDatabases(selectedDsName);
184       if (selectedDsName !== urlDsName) {
185         setSelectedDbName(null);
186         setTables([]);
187       }
188     }
189   }, [selectedDsName]);
190
191   useEffect(() => {
192     if (selectedDsName && selectedDbName) {
193       fetchTables(selectedDsName, selectedDbName);
194     }
195   }, [selectedDsName, selectedDbName]);
196
197   const fetchDatasources = async () => {
198     setDsLoading(true);
199     try {
200       const response = await DSApi.list();
201       if (response.code === 200) {
202         setDatasources(response.data || []);
203       }
204     } catch (error) {
205       console.error('获取数据源列表失败:', error);
206       message.error('获取数据源列表失败');
207     } finally {
208       setDsLoading(false);
209     }
210   };
211
212   const fetchDatabases = async (dsName: string) => {
213     setDbLoading(true);
214     try {
215       const response = await databaseApi.list(dsName);
216       if (response.code === 200) {
217         setDatabases(response.data || []);
218       }
219     } catch (error) {
220       console.error('获取数据库列表失败:', error);
221       message.error('获取数据库列表失败');
222     } finally {
223       setDbLoading(false);
224     }
225   };
226 }
```

```
224     }
225   };
226
227   const fetchTables = async (dsName: string, dbName: string) => {
228     setTableLoading(true);
229     try {
230       const [tableRes, cdcRes] = await Promise.all([
231         tableApi.list(dsName, dbName),
232         listCdcConfigs({ dsName, dbName, syncTool: 'FLINK' })
233       ]);
234
235       let cdcMap = new Map<string, CdcConfigDTO>();
236       if (cdcRes.code === 200 && cdcRes.data) {
237         cdcMap = new Map(cdcRes.data.map(c => [`${c.dbName}.${c.tableName}`, c]));
238       }
239
240       if (tableRes.code === 200) {
241         const tableInfos: TableInfo[] = (tableRes.data || []).map(tbl => {
242           const cdc = cdcMap.get(`${dbName}.${tbl.tableName}`);
243           return {
244             tableName: tbl.tableName,
245             tableComment: tbl.tableComment,
246             cdcEnabled: cdc?.enabled ?? false,
247             cdcConfigId: cdc?.id,
248             envStatus: EnvStatus.SYNCED,
249             updatedAt: new Date().toISOString(),
250           };
251         });
252         setTables(tableInfos);
253       }
254     } catch (error) {
255       console.error('获取表列表失败:', error);
256       message.error('获取表列表失败');
257     } finally {
258       setTableLoading(false);
259     }
260   };
261
262   const handleDsChange = (dsName: string) => {
263     setSelectedDsName(dsName);
264   };
265
266   const handleDbChange = (dbName: string) => {
267     setSelectedDbName(dbName);
268   };
269
270   // CDC 相关操作
271   const handleOpenCdcModal = async (tableName: string, configId?: string) => {
272     setCdcModalTableName(tableName);
273     cdcForm.resetFields();
274     setCdcSelectedSubjectCode('');
275
276     // 加载主题列表
277     try {
278       const subjectList = await listSubjects({parentId: '0'});
279       setSubjects(subjectList || []);
```

```

280     } catch {
281         message.error('加载主题列表失败');
282     }
283
284     if (configId) {
285         // 查看已有配置 (通过 listCdcConfigs 查询, 避免 id/name 混淆)
286         setCdcModalMode('view');
287         setCdcModalConfigName(configId);
288         try {
289             const res = await listCdcConfigs({
290                 dsName: selectedDsName!,
291                 dbName: selectedDbName!,
292                 tableName,
293                 syncTool: 'FLINK',
294             });
295             if (res.code === 200 && res.data && res.data.length > 0) {
296                 const dto = res.data[0];
297                 cdcForm.setFieldsValue({ subjectCode: dto.subjectCode });
298                 setCdcSelectedSubjectCode(dto.subjectCode || '');
299                 setCdcModalConfigEnabled(dto.enabled);
300             }
301         } catch {
302             message.error('加载 CDC 配置失败');
303         }
304     } else {
305         // 创建新配置
306         setCdcModalMode('create');
307         setCdcModalConfigName('');
308         setCdcModalConfigEnabled(false);
309         try {
310             const subjectList = await listSubjects({parentId: '0'});
311             const cdcSubject = subjectList?.find(s => s.subjectCode === 'cdc');
312             if (cdcSubject) {
313                 cdcForm.setFieldsValue({ subjectCode: cdcSubject.subjectCode });
314                 setCdcSelectedSubjectCode(cdcSubject.subjectCode);
315             }
316         } catch {
317             // ignore
318         }
319     }
320
321     setCdcModalVisible(true);
322 };
323
324 const handleCdcSubmit = async () => {
325     if (!selectedDsName || !selectedDbName) return;
326     try {
327         setCdcSubmitting(true);
328
329         if (cdcModalMode === 'view' && cdcModalConfigName) {
330             // view 模式: toggle 启用/停用
331             const res = await listCdcConfigs({
332                 dsName: selectedDsName,
333                 dbName: selectedDbName,
334                 tableName: cdcModalTableName,
335                 syncTool: 'FLINK',

```

```

336     });
337     if (res.code === 200 && res.data && res.data.length > 0) {
338         const dto = res.data[0];
339         const toggleRes = await toggleCdcConfig(dto.name, !dto.enabled);
340         if (toggleRes.code === 200) {
341             message.success(dto.enabled ? '已停用' : '已启用');
342             setCdcModalVisible(false);
343             if (selectedDsName && selectedDbName) {
344                 fetchTables(selectedDsName, selectedDbName);
345             }
346         } else {
347             message.error(toggleRes.message || '操作失败');
348         }
349     }
350 } else {
351     // create 模式: 创建新配置
352     const values = await cdcForm.validateFields();
353     const cmd: CdcConfigCmd = {
354         name: `${selectedDbName}_${cdcModalTableName}_cdc`,
355         dsName: selectedDsName,
356         dbName: selectedDbName,
357         tableName: cdcModalTableName,
358         subjectCode: values.subjectCode,
359         syncTool: 'FLINK',
360         description: `CDC 同步: ${selectedDbName}.${cdcModalTableName}`,
361     };
362
363     const res = await createCdcConfig(cmd);
364     if (res.code === 200) {
365         message.success('CDC 配置创建成功');
366         setCdcModalVisible(false);
367         if (selectedDsName && selectedDbName) {
368             fetchTables(selectedDsName, selectedDbName);
369         }
370     } else {
371         message.error(res.message || '创建失败');
372     }
373 }
374 } catch (error) {
375     console.error('CDC 操作失败:', error);
376 } finally {
377     setCdcSubmitting(false);
378 }
379 };
380
381 const handleToggleCdc = async (record: TableInfo) => {
382     if (!record.cdcConfigId) return;
383     try {
384         const res = await listCdcConfigs({ dsName: selectedDsName!, dbName: selectedDbName
385 e!, tableName: record.tableName, syncTool: 'FLINK' });
386         if (res.code === 200 && res.data && res.data.length > 0) {
387             const dto = res.data[0];
388             const toggleRes = await toggleCdcConfig(dto.name, !record.cdcEnabled);
389             if (toggleRes.code === 200) {
390                 message.success(record.cdcEnabled ? '已停用' : '已启用');
391                 if (selectedDsName && selectedDbName) {

```

```

391         fetchTables(selectedDsName, selectedDbName);
392     }
393     } else {
394         message.error(toggleRes.message || '操作失败');
395     }
396 }
397 } catch (error) {
398     console.error('CDC 启停失败:', error);
399     message.error('操作失败');
400 }
401 };
402
403 const handleCreateTable = () => {
404     setNewTableName('');
405     setCreateModalVisible(true);
406 };
407
408 const handleCreateConfirm = () => {
409     if (!newTableName.trim()) {
410         message.warning('请输入表名');
411         return;
412     }
413     navigate(`/meta/business-ds/table-schema/edit?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${newTableName}&isNew=true`);
414     setCreateModalVisible(false);
415 };
416
417 const handleViewDetail = (tableName: string) => {
418     navigate(`/meta/business-ds/table-schema/detail?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${tableName}`);
419 };
420
421 const handleEdit = (tableName: string) => {
422     navigate(`/meta/business-ds/table-schema/edit?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${tableName}`);
423 };
424
425 const handleDelete = async (tableName: string) => {
426     if (!selectedDsName || !selectedDbName) return;
427
428     try {
429         const response = await tableApi.drop(selectedDsName, selectedDbName, tableName);
430         if (response.code === 200) {
431             message.success('删除成功');
432             fetchTables(selectedDsName, selectedDbName);
433         } else {
434             message.error(response.message || '删除失败');
435         }
436     } catch (error) {
437         console.error('删除表失败:', error);
438         message.error('删除失败');
439     }
440 };
441
442 const handleSyncHistory = async (tableName: string) => {
443     if (!selectedDsName || !selectedDbName) return;

```

```

444
445     try {
446         const response = await tableApi.getSchema(selectedDsName, selectedDbName, tableNam
e);
447         if (response.code === 200 && response.data) {
448             const tableSchema = response.data;
449
450             const importData = {
451                 id: '',
452                 name: tableSchema.tableName,
453                 owner: '',
454                 subjectCode: '',
455                 datasourceType: '',
456                 layerCode: 'ODS',
457                 comment: tableSchema.tableComment || '',
458                 accessCount: '',
459                 lastAccessTime: '',
460                 heatLevel: '',
461                 secretLevel: 'L1',
462                 onlineStatus: 'ONLINE',
463                 createdAt: '',
464                 updatedAt: '',
465                 table: {
466                     catalog: selectedDsName,
467                     schema: selectedDbName,
468                     name: tableSchema.tableName,
469                     comment: tableSchema.tableComment || '',
470                     columns: tableSchema.columns.map(col => ({
471                         name: col.name,
472                         type: convertToIcebergType(col.type),
473                         comment: col.comment || '',
474                         nullable: col.nullable ?? true,
475                         autoIncrement: col.autoIncrement ?? false,
476                         defaultValue: col.defaultValue,
477                         secretLevel: col.secretLevel || 'L1',
478                     })),
479                     indexes: tableSchema.indexes || [],
480                 },
481             };
482
483             navigate('/meta/metadata/metadata_table/edit', {
484                 state: {
485                     mode: 'import',
486                     importData: importData,
487                 }
488             });
489         } else {
490             message.error('获取表结构失败');
491         }
492     } catch (error) {
493         console.error('获取表结构失败:', error);
494         message.error('获取表结构失败');
495     }
496 };
497
498 const getEnvStatusTag = (status: EnvStatus) => {

```

```

499     const config: Record<EnvStatus, { color: string; text: string }> = {
500         [EnvStatus.SYNCED]: {color: 'success', text: '生产已同步'},
501         [EnvStatus.PENDING]: {color: 'warning', text: '待发布'},
502         [EnvStatus.TEST_BEHIND]: {color: 'warning', text: '测试落后'},
503         [EnvStatus.APPROVING]: {color: 'processing', text: '审批中'},
504         [EnvStatus.REJECTED]: {color: 'error', text: '已拒绝'},
505     };
506     const {color, text} = config[status];
507     return <Tag color={color}>{text}</Tag>;
508 };
509
510 const getDatasourceTypeTag = (type: DatasourceType) => {
511     const colorMap: Record<DatasourceType, string> = {
512         [DatasourceType.MYSQL]: 'blue',
513         [DatasourceType.POSTGRESQL]: 'green',
514         [DatasourceType.ICEBERG]: 'purple',
515     };
516     return <Tag color={colorMap[type] || 'default'}>{type}</Tag>;
517 };
518
519 const filteredTables = tables.filter(table =>
520     table.tableName.toLowerCase().includes(searchKeyword.toLowerCase())
521 );
522
523 const columns: ColumnType<TableInfo>[] = [
524     {
525         title: '表名',
526         dataIndex: 'tableName',
527         key: 'tableName',
528         width: 140,
529         render: (name: string) => (
530             <Space>
531                 <TableOutlined style={{ color: '#8B909A' }} />
532                 <span style={{fontWeight: 500, color: '#1D2333'}}>{name}</span>
533             </Space>
534         ),
535     },
536     {
537         title: '描述',
538         dataIndex: 'tableComment',
539         key: 'tableComment',
540         width: 180,
541         ellipsis: true,
542         render: (comment: string) => (
543             <span style={{ color: '#8B909A' }}>{comment || '-'}</span>
544         ),
545     },
546     {
547         title: 'CDC状态',
548         dataIndex: 'cdcEnabled',
549         key: 'cdcEnabled',
550         width: 100,
551         render: (enabled: boolean) => (
552             <Tag color={enabled ? 'success' : 'default'}>
553                 {enabled ? '已启用' : '未启用'}
554             </Tag>

```

```

555     ),
556   },
557   {
558     title: '环境状态',
559     dataIndex: 'envStatus',
560     key: 'envStatus',
561     width: 120,
562     render: (status: EnvStatus) => getEnvStatusTag(status),
563   },
564   {
565     title: '操作',
566     key: 'action',
567     width: 160,
568     fixed: 'right',
569     render: (_, unknown, record: TableInfo) => (
570       <Space size={4}>
571         <Tooltip title="查看详情">
572           <Button
573             type="text"
574             size="small"
575             icon={<EyeOutlined />}
576             onClick={() => handleViewDetail(record.tableName)}
577             style={{ color: '#4F6DF5' }}
578           />
579         </Tooltip>
580         <Tooltip title="编辑">
581           <PermissionButton
582             type="text"
583             size="small"
584             icon={<EditOutlined />}
585             onClick={() => handleEdit(record.tableName)}
586             style={{ color: '#4E5566' }}
587             permission="MENU:meta:business-ds:table:UPDATE"
588           />
589         </Tooltip>
590         <Tooltip title="同步表结构到数仓">
591           <Button
592             type="text"
593             size="small"
594             icon={<HistoryOutlined />}
595             onClick={() => handleSyncHistory(record.tableName)}
596             style={{ color: '#1A9F5C' }}
597           />
598         </Tooltip>
599         <Tooltip title={record.cdcEnabled ? '停用 CDC' : record.cdcConfigId ? '启用
CDC' : '创建 CDC'}>
600           <Button
601             type="text"
602             size="small"
603             icon={<CloudSyncOutlined />}
604             onClick={() => {
605               if (record.cdcEnabled) {
606                 Modal.confirm({
607                   title: '确认停用 CDC 同步',
608                   content: `关闭后 ${record.tableName} 将不再同步数据到数
仓, 是否确认?`,

```

```

609         okText: '确认停用',
610         okType: 'danger',
611         cancelText: '取消',
612         onOk: () => handleToggleCdc(record),
613     });
614     } else if (record.cdcConfigId) {
615         handleOpenCdcModal(record.tableName, record.cdcConfigId);
616     } else {
617         handleOpenCdcModal(record.tableName);
618     }
619     }}
620     style={{ color: record.cdcEnabled ? '#722ED1' : '#8B909A' }}
621     />
622 </Tooltip>
623 <Popconfirm
624     title="确定删除该表吗？此操作不可恢复！"
625     onConfirm={() => handleDelete(record.tableName)}
626     okText="确定"
627     cancelText="取消"
628     >
629     <Tooltip title="删除">
630         <PermissionButton
631             type="text"
632             size="small"
633             danger
634             icon={<DeleteOutlined />}
635             permission="MENU:meta:business-ds:table:DELETE"
636         />
637     </Tooltip>
638 </Popconfirm>
639 </Space>
640     ),
641 },
642 ];
643
644 return (
645     <div className="page-container">
646         { /* 页面标题区 */ }
647         <div className="page-header-section" style={{ marginBottom: 20 }}>
648             <div className="page-title-wrapper">
649                 <Title level={4} className="page-title">表结构管理</Title>
650                 <Text className="page-subtitle">
651                     浏览和管理数据库表结构，支持表结构同步到数据仓库
652                 </Text>
653             </div>
654             <PermissionButton
655                 type="primary"
656                 icon={<PlusOutlined />}
657                 onClick={handleCreateTable}
658                 disabled={!selectedDsName || !selectedDbName}
659                 size="middle"
660                 permission="MENU:meta:business-ds:table:CREATE"
661             >
662                 新建表
663             </PermissionButton>
664         </div>

```

```

665
666      { /* 搜索筛选区 */ }
667      <Card
668          style={{
669              marginBottom: 16,
670              borderRadius: 12,
671              boxShadow: '0 2px 8px rgba(29, 35, 51, 0.03)',
672          }}
673          styles={{ body: { padding: '16px 20px' } }}
674      >
675          <Row gutter={[16, 12]} align="middle">
676              <Col>
677                  <Space>
678                      <DatabaseOutlined style={{ color: '#8B909A' }} />
679                      <Text type="secondary" style={{ fontSize: 13 }}>数据源</Text>
680                      <Select
681                          style={{ width: 200 }}
682                          placeholder="请选择数据源"
683                          value={selectedDsName}
684                          onChange={handleDsChange}
685                          loading={dsLoading}
686                          showSearch
687                          optionFilterProp="label"
688                          suffixIcon={<DatabaseOutlined style={{ color: '#8B909A', fontS
689                              size: 12 }} />}
690                      >
691                          {datasources.map(ds => (
692                              <Select.Option key={ds.name} value={ds.name} label={ds.name}
693                                  >
694                                      <Space>
695                                          {getDatasourceTypeTag(ds.datasourceType)}
696                                          {ds.name}
697                                      </Space>
698                                  </Select.Option>
699                              ))}
700                      </Select>
701                  </Space>
702              </Col>
703              <Col>
704                  <Space>
705                      <Text type="secondary" style={{ fontSize: 13 }}>数据库</Text>
706                      <Select
707                          style={{ width: 180 }}
708                          placeholder="请选择数据库"
709                          value={selectedDbName}
710                          onChange={handleDbChange}
711                          loading={dbLoading}
712                          disabled={!selectedDsName}
713                          showSearch
714                          optionFilterProp="children"
715                      >
716                          {databases.map(db => (
717                              <Select.Option key={db.name} value={db.name}
718                                  >
                                  {db.name}
                                  </Select.Option>
                              ))}

```

```

719             </Select>
720         </Space>
721     </Col>
722     <Col flex="auto">
723         <Input.Search
724             placeholder="搜索表名"
725             value={searchKeyword}
726             onChange={e => setSearchKeyword(e.target.value)}
727             style={{ width: 240 }}
728             allowClear
729             prefix={<SearchOutlined style={{ color: '#8B909A' }} />}
730         />
731     </Col>
732     <Col>
733         <Text type="secondary" style={{ fontSize: 13 }}>
734             共 <Text strong style={{ color: '#4F6DF5' }}>{filteredTables.length
h}</Text> 个表
735         </Text>
736     </Col>
737 </Row>
738 </Card>
739
740 {/* 数据表格 */}
741 <Card
742     style={{
743         borderRadius: 12,
744         boxShadow: '0 2px 8px rgba(29, 35, 51, 0.03)',
745         overflow: 'hidden',
746     }}
747     styles={{ body: { padding: 0 } }}
748 >
749     <Table
750         columns={columns}
751         dataSource={filteredTables}
752         rowKey="tableName"
753         loading={tableLoading}
754         pagination={{
755             pageSize: 10,
756             showSizeChanger: true,
757             showTotal: (total) => `共 ${total} 条`,
758         }}
759         locale={{emptyText: selectedDsName && selectedDbName ? '暂无数据' : '请先选
择数据源和数据库'}}
760         scroll={{ x: 700 }}
761     />
762 </Card>
763
764 {/* 新建表模态框 */}
765 <Modal
766     title="新建表"
767     open={createModalVisible}
768     onCancel={() => setCreateModalVisible(false)}
769     onOk={handleCreateConfirm}
770     width={400}
771 >
772     <Form layout="vertical" style={{ marginTop: 8 }}>

```

```

773         <Form.Item label="表名" required>
774             <Input
775                 placeholder="请输入表名"
776                 value={newTableName}
777                 onChange={e => setNewTableName(e.target.value)}
778             />
779         </Form.Item>
780     </Form>
781 </Modal>
782
783     { /* CDC 配置模态框 */ }
784     <Modal
785         title={cdcModalMode === 'create' ? `创建 CDC 同步 - ${cdcModalTableName}` : `CD
C 同步 - ${cdcModalTableName}`}
786         open={cdcModalVisible}
787         onOk={handleCdcSubmit}
788         onCancel={() => setCdcModalVisible(false)}
789         confirmLoading={cdcSubmitting}
790         okText={cdcModalMode === 'view' ? (cdcModalConfigEnabled ? '停用' : '启用') :
'确定'}
791         width={520}
792     >
793         <Form
794             form={cdcForm}
795             layout="vertical"
796             style={{ marginTop: 16 }}
797         >
798             <Form.Item label="数据源">
799                 <Input value={selectedDsName || ''} disabled />
800             </Form.Item>
801             <Form.Item label="数据库">
802                 <Input value={selectedDbName || ''} disabled />
803             </Form.Item>
804             <Form.Item label="表名">
805                 <Input value={cdcModalTableName} disabled />
806             </Form.Item>
807             <Form.Item
808                 name="subjectCode"
809                 label="主题"
810                 rules={[{ required: true, message: '请选择主题' }]}
811             >
812                 <Select
813                     placeholder="请选择主题"
814                     showSearch
815                     optionFilterProp="children"
816                     onChange={(value) => setCdcSelectedSubjectCode(value as string)}
817                 >
818                     {subjects.map(subject => (
819                         <Select.Option key={subject.subjectCode} value={subject.subjectCode}
820                             {subject.subjectName} ({subject.subjectCode})
821                         </Select.Option>
822                     ))}
823                 </Select>
824             </Form.Item>
825             {cdcSelectedSubjectCode && selectedDbName && (

```

```

826         <Form.Item label="目标 ODS 表名">
827             <Input
828                 value={`ods_cdc_raw_${cdcSelectedSubjectCode.replace(/^[a-zA-Z
0-9_]/g, '_')}_${selectedDbName.replace(/^[a-zA-Z0-9_]/g, '_')}_${cdcModalTableName.replace(/^[a-zA-Z
0-9_]/g, '_')}`}
829                 disabled
830             />
831         </Form.Item>
832     )}
833 </Form>
834 </Modal>
835 </div>
836 );
837 };
838
839 export default TableSchemaManagement;
840
841
842 /* ===== */
843 /* 文件: src/pages/bi/dashboard/DashboardEditor.tsx */
844 /* 软件名称: Cyan数据中台 */
845 /* 版本号: V1.0 */
846 /* 著作权人: 闫晨阳 */
847 /* ===== */
848 import React, { useEffect, useState, useCallback, useRef } from 'react';
849 import { Button, Input, Space, message, Spin, Empty } from 'antd';
850 import { SaveOutlined, ArrowLeftOutlined, EyeOutlined, MenuFoldOutlined, MenuUnfoldOutlined }
from '@ant-design/icons';
851 import { useNavigate, useParams } from 'react-router-dom';
852 import { dashboardApi, chartApi, DashboardCmd, ChartDTO, ChartRef, ChartDataDTO, ChartType, Me
tricBiAnalysisCmd, FilterOperator, AggregateType } from '@api/DatabiApi';
853 import GridLayoutImport, { Layout } from 'react-grid-layout';
854 const GridLayout = GridLayoutImport as any;
855 import 'react-grid-layout/css/styles.css';
856 import 'react-resizable/css/styles.css';
857 import ChartLibrary from './components/ChartLibrary';
858 import PanelCard from './components/PanelCard';
859 import PanelOptions from './components/PanelOptions';
860 import FilterChart from './components/FilterChart';
861 import EChartsChart from '@pages/bi/components/EChartsChart';
862
863 const GRID_COLS = 12;
864 const ROW_HEIGHT = 50;
865 const SIDEBAR_WIDTH = 260;
866 const DEFAULT_W = 6;
867 const DEFAULT_H = 5;
868
869 const isFilterChartType = (chartType?: ChartType) => {
870     return chartType === ChartType.FILTER_SELECT || chartType === ChartType.FILTER_MULTI ||
871         chartType === ChartType.FILTER_DATE || chartType === ChartType.FILTER_DATE_RANGE;
872 };
873
874 // 找第一个不重叠的位置
875 const findFreePosition = (refs: ChartRef[], w: number, h: number): { x: number; y: number } =>
{
876     for (let y = 0; y < 100; y += h) {

```

```

877     for (let x = 0; x + w <= GRID_COLS; x += w) {
878         const overlap = refs.some(ref =>
879             ref.x < x + w && ref.x + ref.w > x &&
880             ref.y < y + h && ref.y + ref.h > y
881         );
882         if (!overlap) return { x, y };
883     }
884 }
885 return { x: 0, y: Math.max(0, ...refs.map(r => r.y + r.h), 0) };
886 };
887
888 // 检测同一行宽度是否溢出
889 const checkRowOverflow = (refs: ChartRef[]): string | null => {
890     const rows: Record<number, number> = {};
891     for (const ref of refs) {
892         for (let y = ref.y; y < ref.y + ref.h; y++) {
893             rows[y] = (rows[y] || 0) + ref.w;
894         }
895     }
896     for (const [y, total] of Object.entries(rows)) {
897         if (total > GRID_COLS) {
898             return `第 ${Number(y) + 1} 行图表总宽度(${total})超出限制(${GRID_COLS}), 请调整布局`;
899         }
900     }
901     return null;
902 };
903
904 const DashboardEditor: React.FC = () => {
905     const navigate = useNavigate();
906     const { id } = useParams<{ id?: string }>();
907     const isEdit = !!id;
908
909     const [name, setName] = useState('');
910     const [description, setDescription] = useState('');
911     const [chartRefs, setChartRefs] = useState<ChartRef[]>([]);
912     const [charts, setCharts] = useState<ChartDTO[]>([]);
913     const [chartDataMap, setChartDataMap] = useState<Record<string, ChartDataDTO>>({});
914     const [chartLoadingMap, setChartLoadingMap] = useState<Record<string, boolean>>({});
915     const [filterValuesMap, setFilterValuesMap] = useState<Record<string, string[]>>({});
916     const filterValuesMapRef = useRef<Record<string, string[]>>({});
917     const [, setLoading] = useState(false);
918     const [saving, setSaving] = useState(false);
919     const originalDslMap = useRef<Record<string, MetricBiAnalysisCmd>>({});
920
921     useEffect(() => {
922         filterValuesMapRef.current = filterValuesMap;
923     }, [filterValuesMap]);
924     const [selectedIndex, setSelectedIndex] = useState<number | null>(null);
925     const [layout, setLayout] = useState<{ i: string; x: number; y: number; w: number; h: number }[]>([]);
926     const [leftCollapsed, setLeftCollapsed] = useState(false);
927     const [rightCollapsed, setRightCollapsed] = useState(false);
928     const [canvasWidth, setCanvasWidth] = useState(960);
929
930     // 加载
931     useEffect(() => {

```

```

932     chartApi.list().then(res => { if (res.code === 200) setCharts(res.data); });
933     if (isEdit && id) {
934         setLoading(true);
935         dashboardApi.getById(id)
936             .then(res => {
937                 if (res.code === 200) {
938                     setName(res.data.name);
939                     setDescription(res.data.description || '');
940                     const refs = res.data.chartRefs || [];
941                     setChartRefs(refs);
942                     setLayout(refs.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.
y, w: ref.w, h: ref.h })));
943                     executeAllCharts(refs);
944                 } else {
945                     message.error(res.message || '加载看板失败');
946                 }
947             })
948             .catch(() => message.error('加载看板失败'))
949             .finally(() => setLoading(false));
950     }
951 }, [isEdit, id]);
952
953 useEffect(() => {
954     const updateWidth = () => {
955         setCanvasWidth(Math.max(600, window.innerWidth - (leftCollapsed ? 0 : SIDEBAR_WIDT
H) - (rightCollapsed ? 0 : SIDEBAR_WIDTH) - 64));
956     };
957     updateWidth();
958     window.addEventListener('resize', updateWidth);
959     return () => window.removeEventListener('resize', updateWidth);
960 }, [leftCollapsed, rightCollapsed]);
961
962 // 构建合并后的 DSL (级联筛选条件)
963 const buildMergedDsl = useCallback((chartId: string): MetricBiAnalysisCmd | null => {
964     const originalDsl = originalDslMap.current[chartId];
965     if (!originalDsl) return null;
966     const ref = chartRefs.find(r => r.chartId === chartId);
967     if (!ref) return originalDsl;
968     // 收集所有影响该图表的筛选器来源:
969     // 1. 图表自身的 cascadeFrom (正向配置)
970     // 2. 哪些筛选器的 cascadeFrom 包含该图表 (反向配置)
971     const sourceIds = new Set<string>(ref.cascadeFrom || []);
972     chartRefs.forEach(filterRef => {
973         const filterChart = charts.find(c => c.id === filterRef.chartId);
974         if (filterChart && isFilterChartType(filterChart.chartType) && (filterRef.cascadeF
rom || []).includes(chartId)) {
975             sourceIds.add(filterRef.chartId);
976         }
977     });
978     const newFilters = [...(originalDsl.filters || [])];
979     for (const sourceChartId of sourceIds) {
980         const sourceChart = charts.find(c => c.id === sourceChartId);
981         if (!sourceChart || !isFilterChartType(sourceChart.chartType)) continue;
982         const values = filterValuesMapRef.current[sourceChartId] || [];
983         const dimCode = sourceChart.metricAnalysisCmd?.dimensions?.[0]?.dimCode || '';
984         if (!dimCode) continue;

```

```

985         if (values.length === 0) {
986             const idx = newFilters.findIndex(f => f.dimCode === dimCode);
987             if (idx >= 0) newFilters.splice(idx, 1);
988             continue;
989         }
990         let operator: FilterOperator;
991         if (sourceChart.chartType === ChartType.FILTER_DATE_RANGE) operator = FilterOperat
or.BETWEEN;
992         else if (values.length > 1) operator = FilterOperator.IN;
993         else operator = FilterOperator.EQ;
994         const idx = newFilters.findIndex(f => f.dimCode === dimCode);
995         if (idx >= 0) newFilters[idx] = { dimCode, operator, values };
996         else newFilters.push({ dimCode, operator, values });
997     }
998     return { ...originalDsl, filters: newFilters };
999 }, [charts, chartRefs]);
1000
1001 // 执行单个图表 (支持传入自定义 DSL)
1002 const executeChart = useCallback(async (chartId: string, dsl?: MetricBiAnalysisCmd) => {
1003     setChartLoadingMap(prev => ({ ...prev, [chartId]: true }));
1004     try {
1005         const body = dsl ? { metricAnalysisCmd: dsl } : undefined;
1006         const res = await chartApi.execute(chartId, body);
1007         if (res.code === 200 && res.data.status === 'SUCCESS') {
1008             setChartDataMap(prev => ({ ...prev, [chartId]: res.data }));
1009         }
1010     } catch { /* ignore */ }
1011     finally {
1012         setChartLoadingMap(prev => ({ ...prev, [chartId]: false }));
1013     }
1014 }, []);
1015
1016 // 执行所有图表 (无维度日期筛选框跳过 execute, 同时保存原始 DSL)
1017 const executeAllCharts = async (refs: ChartRef[]) => {
1018     const loadingMap: Record<string, boolean> = {};
1019     refs.forEach(r => loadingMap[r.chartId] = true);
1020     setChartLoadingMap(loadingMap);
1021     const dataMap: Record<string, ChartDataDTO> = {};
1022     const dslMap: Record<string, MetricBiAnalysisCmd> = {};
1023     await Promise.all(refs.map(async ref => {
1024         const chart = charts.find(c => c.id === ref.chartId);
1025         if (chart?.metricAnalysisCmd) {
1026             dslMap[ref.chartId] = JSON.parse(JSON.stringify(chart.metricAnalysisCmd));
1027         }
1028         const dimCode = chart?.metricAnalysisCmd?.dimensions?.[0]?.dimCode;
1029         const isDate = chart?.chartType === ChartType.FILTER_DATE || chart?.chartType ===
ChartType.FILTER_DATE_RANGE;
1030         if (isDate && !dimCode) return;
1031         try {
1032             const res = await chartApi.execute(ref.chartId);
1033             if (res.code === 200 && res.data.status === 'SUCCESS') dataMap[ref.chartId] =
res.data;
1034         } catch { /* ignore */ }
1035     }));
1036     originalDslMap.current = dslMap;
1037     setChartDataMap(dataMap);

```

```

1038     setChartLoadingMap({});
1039 };
1040
1041 // 筛选框值变化回调
1042 const handleFilterChange = useCallback((chartId: string, values: string[]) => {
1043     setFilterValuesMap(prev => ({ ...prev, [chartId]: values }));
1044     const sourceChart = charts.find(c => c.id === chartId);
1045     const isSourceFilter = sourceChart ? isFilterChartType(sourceChart.chartType) : false;
1046     // 查找受影响图表：
1047     // 1. 图表的 cascadeFrom 包含当前筛选器（正向配置）
1048     // 2. 当前筛选器的 cascadeFrom 包含该图表，且该图表不是筛选器（反向配置）
1049     const sourceRef = chartRefs.find(r => r.chartId === chartId);
1050     const affectedRefs = chartRefs.filter(ref => {
1051         const forward = (ref.cascadeFrom || []).includes(chartId);
1052         const targetChart = charts.find(c => c.id === ref.chartId);
1053         const reverse = isSourceFilter && targetChart && !isFilterChartType(targetChart.ch
1054 artType) && (sourceRef?.cascadeFrom || []).includes(ref.chartId);
1055         return forward || reverse;
1056     });
1057     if (affectedRefs.length === 0) return;
1058     const timer = setTimeout(() => {
1059         affectedRefs.forEach(ref => {
1060             const mergedDsl = buildMergedDsl(ref.chartId);
1061             if (mergedDsl) executeChart(ref.chartId, mergedDsl);
1062         });
1063     }, 300);
1064     return () => clearTimeout(timer);
1065 }, [charts, chartRefs, buildMergedDsl, executeChart]);
1066
1067 const handleLayoutChange = useCallback((newLayout: Layout) => {
1068     setLayout(newLayout as { i: string; x: number; y: number; w: number; h: number }[]);
1069     const nextRefs = chartRefs.map((ref, idx) => {
1070         const l = newLayout.find(item => item.i === String(idx));
1071         return l ? { ...ref, x: l.x, y: l.y, w: l.w, h: l.h } : ref;
1072     });
1073     setChartRefs(nextRefs);
1074     // 布局变化后检测溢出
1075     const overflowMsg = checkRowOverflow(nextRefs);
1076     if (overflowMsg) message.warning(overflowMsg);
1077 }, [chartRefs]);
1078
1079 const handleAddChart = (chartId: string) => {
1080     if (chartRefs.find(c => c.chartId === chartId)) {
1081         message.warning('该图表已添加');
1082         return;
1083     }
1084     const pos = findFreePosition(chartRefs, DEFAULT_W, DEFAULT_H);
1085     const newRef: ChartRef = { chartId, x: pos.x, y: pos.y, w: DEFAULT_W, h: DEFAULT_H, ti
1086 tleVisible: true, borderStyle: 'default', cascadeFrom: [] };
1087     const nextRefs = [...chartRefs, newRef];
1088     setChartRefs(nextRefs);
1089     setLayout(nextRefs.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w,
1090 h: ref.h })));
1091     setSelectedIndex(nextRefs.length - 1);
1092     // 执行新图表（无维度日期筛选框跳过，同时保存原始 DSL）
1093     const chart = charts.find(c => c.id === chartId);

```

```

1091     if (chart?.metricAnalysisCmd) {
1092         originalDslMap.current[chartId] = JSON.parse(JSON.stringify(chart.metricAnalysisCmd));
1093     }
1094     const dimCode = chart?.metricAnalysisCmd?.dimensions?.[0]?.dimCode;
1095     const isDate = chart?.chartType === ChartType.FILTER_DATE || chart?.chartType === ChartType.FILTER_DATE_RANGE;
1096     if (!(isDate && !dimCode)) {
1097         setChartLoadingMap(p => ({ ...p, [chartId]: true }));
1098         chartApi.execute(chartId).then(res => {
1099             if (res.code === 200 && res.data.status === 'SUCCESS') setChartDataMap(p => ({
1100                 ...p, [chartId]: res.data }));
1101             setChartLoadingMap(p => ({ ...p, [chartId]: false }));
1102         });
1103     };
1104
1105     const handleDuplicateChart = (index: number) => {
1106         const ref = chartRefs[index];
1107         const pos = findFreePosition(chartRefs, ref.w, ref.h);
1108         const newRef: ChartRef = { ...ref, x: pos.x, y: pos.y };
1109         const nextRefs = [...chartRefs, newRef];
1110         setChartRefs(nextRefs);
1111         setLayout(nextRefs.map((r, idx) => ({ i: String(idx), x: r.x, y: r.y, w: r.w, h: r.h })));
1112         setSelectedIndex(nextRefs.length - 1);
1113     };
1114
1115     const handleRemoveChart = (index: number) => {
1116         const next = [...chartRefs];
1117         const removedId = next[index]?.chartId;
1118         next.splice(index, 1);
1119         setChartRefs(next);
1120         setLayout(next.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w, h: ref.h })));
1121         if (removedId) setChartDataMap(p => { const m = { ...p }; delete m[removedId]; return m; });
1122         if (selectedIndex === index) setSelectedIndex(null);
1123         else if (selectedIndex !== null && selectedIndex > index) setSelectedIndex(selectedIndex - 1);
1124     };
1125
1126     const handleUpdateRef = (index: number, key: keyof ChartRef, value: unknown) => {
1127         const next = [...chartRefs];
1128         next[index] = { ...next[index], [key]: value };
1129         setChartRefs(next);
1130         if (key === 'w' || key === 'h') {
1131             setLayout(next.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w, h: ref.h })));
1132         }
1133     };
1134
1135     const handleSave = async (andPreview = false) => {
1136         if (!name.trim()) { message.warning('请输入看板名称'); return null; }
1137         const overflowMsg = checkRowOverflow(chartRefs);
1138         if (overflowMsg) { message.warning(`布局有误: ${overflowMsg}, 请修正后再保存`); return null; }

```

```

1; }
1139     const cmd: DashboardCmd = {
1140         name: name.trim(), description: description || undefined,
1141         chartRefs: chartRefs.length ? chartRefs : undefined,
1142         layoutConfig: JSON.stringify({ cols: GRID_COLS, rowHeight: ROW_HEIGHT, compact: true, version: 2 }),
1143     };
1144     setSaving(true);
1145     try {
1146         let res;
1147         if (isEdit && id) res = await dashboardApi.update(id, cmd);
1148         else res = await dashboardApi.create(cmd);
1149         if (res.code === 200) {
1150             message.success(isEdit ? '更新成功' : '创建成功');
1151             const dashboardId = isEdit && id ? id : String(res.data);
1152             if (andPreview) navigate(`/bi/dashboard/view/${dashboardId}`);
1153             else if (!isEdit) navigate(`/bi/dashboard/edit/${dashboardId}`);
1154             return dashboardId;
1155         } else { message.error(res.message || '保存失败'); return null; }
1156     } catch { message.error('保存失败'); return null; }
1157     finally { setSaving(false); }
1158 };
1159
1160 const handlePreview = async () => {
1161     if (isEdit && id) navigate(`/bi/dashboard/view/${id}`);
1162     else { const did = await handleSave(true); if (!did) message.warning('请先保存看板'); }
1163 };
1164
1165 const existingChartIds = chartRefs.map(r => r.chartId);
1166
1167 // 渲染面板内容
1168 const renderContent = (chartId: string, chart?: ChartDT0) => {
1169     const isLoading = chartLoadingMap[chartId];
1170     const data = chartDataMap[chartId];
1171     if (isLoading) return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><Spin size="small" /></div>;
1172     if (!data || data.status !== 'SUCCESS') return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><Empty description="无预览数据" imageStyle={{ height: 40 }} /></div>;
1173     if (!chart) return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%', color: '#d9d9d9' }}>未知图表</div>;
1174     if (isFilterChartType(chart.chartType)) {
1175         return (
1176             <div style={{ padding: '8px 12px', height: '100%' }}>
1177                 <FilterChart chart={chart} data={data} onChange={(vals) => handleFilterChange(chart.id, vals)} />
1178             </div>
1179         );
1180     }
1181     if (chart.chartType === ChartType.NUMBER && chart.metricAnalysisCmd?.metrics?.length) {
1182         return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><div style={{ textAlign: 'center' }}><div style={{ fontSize: 28, fontWeight: 600, color: '#4F6DF5' }}>{Number(data.rows[0]?.[chart.metricAnalysisCmd.metrics[0].metricCode] ?? 0).toLocaleString()}</div><div style={{ fontSize: 12, color: '#999' }}>{chart.metricAnalysisCmd.metrics[0].metricName || chart.metricAnalysisCmd.metrics[0].alias || chart.metricAnalysisCmd.metrics[0].met

```

```

ricCode}</div></div></div>;
1183         }
1184         if (chart.chartType === ChartType.TABLE) {
1185             const aliasMap: Record<string, string> = {};
1186             chart.metricAnalysisCmd?.dimensions?.forEach(d => { aliasMap[d.dimCode] = d.dimName || d.alias || d.dimCode; });
1187             chart.metricAnalysisCmd?.metrics?.forEach(m => { aliasMap[m.metricCode] = m.metricName || m.alias || m.metricCode; });
1188             return <div style={{ height: '100%', overflow: 'auto' }}><table style={{ width: '100%', fontSize: 11, borderCollapse: 'collapse' }}><thead><tr>{data.columns.map(col => <th key={col} style={{ borderBottom: '1px solid #f0f0f0', padding: '3px 6px', textAlign: 'left', fontWeight: 500, background: '#fafafa', fontSize: 10 }}>{aliasMap[col] || col}</th>)}</tr></thead><tbody>{data.rows.slice(0, 20).map((row, idx) => <tr key={idx}>{data.columns.map(col => <td key={col} style={{ borderBottom: '1px solid #f0f0f0', padding: '3px 6px' }}>{String(row[col] ?? '')}</td>)}</tr>)}</tbody></table></div>;
1189         }
1190         if (data.rows.length > 0) {
1191             return <div style={{ width: '100%', height: '100%' }}><EChartsChart chartType={chart.chartType} columns={data.columns} rows={data.rows} dimensions={chart.metricAnalysisCmd?.dimensions?.map(d => ({ field: d.dimCode, alias: d.dimName || d.alias || d.dimCode })) || []} metrics={chart.metricAnalysisCmd?.metrics?.map(m => ({ field: m.metricCode, aggregate: AggregateType.SUM, alias: m.metricName || m.alias || m.metricCode })) || []} /></div>;
1192         }
1193         return <Empty description="暂无数据" imageStyle={{ height: 40 }} />;
1194     };
1195
1196     return (
1197         <div style={{ display: 'flex', flexDirection: 'column', height: '100%', background: '#f0f2f5' }}>
1198             { /* 顶部工具栏 */ }
1199             <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '8px 16px', borderBottom: '1px solid #e8e8e8', background: '#fff', flexShrink: 0 }}>
1200                 <Space>
1201                     <Button icon={<ArrowLeftOutlined />} size="small" onClick={() => navigate('/bi/dashboard')}>返回</Button>
1202                     <Input placeholder="看板名称" value={name} onChange={e => setName(e.target.value)} style={{ width: 200 }} variant="borderless" />
1203                     <Input placeholder="描述 (可选)" value={description} onChange={e => setDescription(e.target.value)} style={{ width: 240 }} variant="borderless" />
1204                 </Space>
1205                 <Space>
1206                     <Button icon={<EyeOutlined />} size="small" onClick={handlePreview}>预览</Button>
1207                     <Button type="primary" icon={<SaveOutlined />} size="small" loading={saving} onClick={() => handleSave()}>保存</Button>
1208                 </Space>
1209             </div>
1210
1211             { /* 主体 */ }
1212             <div style={{ display: 'flex', flex: 1, overflow: 'hidden' }}>
1213                 { /* 左侧 */ }
1214                 <div style={{ width: leftCollapsed ? 40 : SIDEBAR_WIDTH, flexShrink: 0, borderRight: '1px solid #e8e8e8', background: '#fff', overflow: 'hidden', display: 'flex', flexDirection: 'column', transition: 'width 0.2s' }}>
1215                     {leftCollapsed ? (
1216                         <div style={{ display: 'flex', justifyContent: 'center', padding: '12px' }}>

```

```

x 0', cursor: 'pointer' }} onClick={() => setLeftCollapsed(false)}><MenuUnfoldOutlined style={{ color: '#666' }} /></div>
1217         ) : (
1218             <>
1219                 <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px 12px', borderBottom: '1px solid #f0f0f0' }}>
1220                     <span style={{ fontSize: 13, fontWeight: 500 }}>组件库</span>
1221                     <MenuFoldOutlined style={{ color: '#999', cursor: 'pointer' }}
onClick={() => setLeftCollapsed(true)} />
1222                 </div>
1223                 <div style={{ flex: 1, overflow: 'hidden' }}>
1224                     <ChartLibrary charts={charts} onAddChart={handleAddChart} existingChartIds={existingChartIds} onRefreshCharts={() => chartApi.list().then(res => { if (res.code == 200) setCharts(res.data); })} />
1225                 </div>
1226             </>
1227         )}
1228     </div>
1229
1230     { /* 画布 */ }
1231     <div style={{ flex: 1, overflow: 'auto', padding: 16, background: '#f0f2f5' }}>
>
1232         <div style={{ background: '#fff', minHeight: 600, border: '1px dashed #d9d9d9', position: 'relative', padding: 8 }}>
1233             {chartRefs.length === 0 ? (
1234                 <div style={{ display: 'flex', alignItems: 'center', justify-content: 'center', height: 400, color: '#999', fontSize: 14, border: '2px dashed #e8e8e8', borderRadius: 8, margin: 16 }}>请从左侧组件库添加图表到画布</div>
1235             ) : (
1236                 <GridLayout className="layout" layout={layout as any} cols={GRID_COLS} rowHeight={ROW_HEIGHT} width={canvasWidth} onLayoutChange={handleLayoutChange} margin={[12, 12]} isBounded={false}>
1237                     {chartRefs.map((ref, idx) => {
1238                         const chart = charts.find(c => c.id === ref.chartId);
1239                         const isSelected = selectedIndex === idx;
1240                         return (
1241                             <div key={String(idx)}>
1242                                 <PanelCard title={chart?.name || ref.chartId} selected={isSelected} bgColor={ref.bgColor || undefined} borderStyle={ref.borderStyle} titleVisible={ref.titleVisible} onClick={() => setSelectedIndex(idx)} onDuplicate={() => handleDuplicateChart(idx)} onDelete={() => handleRemoveChart(idx)} dragHandleProps={{ className: 'drag-handle' }}>
1243                                     {renderContent(ref.chartId, chart)}
1244                                 </PanelCard>
1245                             </div>
1246                         );
1247                     })}
1248                 </GridLayout>
1249             )}
1250         </div>
1251     </div>
1252
1253     { /* 右侧 */ }
1254     <div style={{ width: rightCollapsed ? 40 : SIDEBAR_WIDTH, flexShrink: 0, borderLeft: '1px solid #e8e8e8', background: '#fff', overflow: 'hidden', display: 'flex', flexDirection: 'column', transition: 'width 0.2s' }}>
1255         {rightCollapsed ? (

```

```

1256             <div style={{ display: 'flex', justifyContent: 'center', padding: '12p
x 0', cursor: 'pointer' }} onClick={() => setRightCollapsed(false)}><MenuUnfoldOutlined style={{ col
or: '#666' }} /></div>
1257             ) : (
1258                 <>
1259                     <div style={{ display: 'flex', justifyContent: 'space-between', al
ignKeyItems: 'center', padding: '10px 12px', borderBottom: '1px solid #f0f0f0' }}>
1260                         <span style={{ fontSize: 13, fontWeight: 500 }}>属性设置</span>
1261                         <MenuFoldOutlined style={{ color: '#999', cursor: 'pointer' }}
onClick={() => setRightCollapsed(true)} />
1262                     </div>
1263                     <div style={{ flex: 1, overflow: 'auto' }}>
1264                         {selectedIndex !== null && chartRefs[selectedIndex] ? (
1265                             <PanelOptions chartRef={chartRefs[selectedIndex]} chart={c
harts.find(c => c.id === chartRefs[selectedIndex].chartId)} allChartRefs={chartRefs} allCharts={char
ts} onChange={(key, value) => handleUpdateRef(selectedIndex, key as keyof ChartRef, value)} />
1266                         ) : (
1267                             <div style={{ color: '#999', textAlign: 'center', paddingT
op: 60, fontSize: 13 }}>点击画布中的面板<br />进行配置</div>
1268                         )}
1269                     </div>
1270                 </>
1271             )}
1272         </div>
1273     </div>
1274 </div>
1275 );
1276 };
1277
1278 export default DashboardEditor;
1279
1280
1281 /* ===== */
1282 /* 文件: src/pages/auth/role/index.tsx */
1283 /* 软件名称: Cyan数据中台 */
1284 /* 版本号: V1.0 */
1285 /* 著作权人: 闫晨阳 */
1286 /* ===== */
1287 import React, { useEffect, useState } from 'react';
1288 import {
1289     Table,
1290     Button,
1291     Drawer,
1292     Form,
1293     Input,
1294     Space,
1295     Tag,
1296     message,
1297     Spin,
1298     Empty,
1299     Modal,
1300     Transfer,
1301     TreeSelect,
1302     Typography,
1303     Select,
1304 } from 'antd';

```

```

1305 import {
1306     PlusOutlined,
1307     EditOutlined,
1308     DeleteOutlined,
1309     TeamOutlined,
1310 } from '@ant-design/icons';
1311 import PermissionButton from '@component/permission/PermissionButton';
1312 import {
1313     listRoles,
1314     saveRole,
1315     deleteRole,
1316     listRoleMembers,
1317     addRoleMembers,
1318     removeRoleMember,
1319     RoleDTO,
1320     RoleCmd,
1321     RoleMemberDTO,
1322     getUserFunctionPermissions,
1323     FunctionPermissionNode,
1324 } from '@api/DataAuthApi';
1325 import { listEmployees } from '@api/EmployeeApi';
1326
1327 const { Title } = Typography;
1328
1329 interface TreeDataNode {
1330     title: string;
1331     value: string;
1332     children?: TreeDataNode[];
1333 }
1334
1335 const convertPermissionTree = (nodes: FunctionPermissionNode[]): TreeDataNode[] => {
1336     return nodes.map(node => ({
1337         title: node.name,
1338         value: node.key,
1339         children: node.children ? convertPermissionTree(node.children) : undefined,
1340     }));
1341 };
1342
1343 const RolePage: React.FC = () => {
1344     const [roles, setRoles] = useState<RoleDTO[]>([]);
1345     const [loading, setLoading] = useState(false);
1346     const [drawerVisible, setDrawerVisible] = useState(false);
1347     const [editingRole, setEditingRole] = useState<RoleDTO | null>(null);
1348     const [memberModalVisible, setMemberModalVisible] = useState(false);
1349     const [currentRoleId, setCurrentRoleId] = useState<string>('');
1350     const [members, setMembers] = useState<RoleMemberDTO[]>([]);
1351     const [memberLoading, setMemberLoading] = useState(false);
1352     const [form] = Form.useForm();
1353     const [memberFormVisible, setMemberFormVisible] = useState(false);
1354     const [targetKeys, setTargetKeys] = useState<string[]>([]);
1355     const [allUsers, setAllUsers] = useState<RoleMemberDTO[]>([]);
1356     const [functionPermissionTree, setFunctionPermissionTree] = useState<TreeDataNode[]>([]);
1357
1358     const fetchRoles = async () => {
1359         setLoading(true);
1360         try {

```

```

1361         const res = await listRoles();
1362         if (res.code === 200 && res.data) {
1363             setRoles(res.data);
1364         }
1365     } catch {
1366         message.error('获取角色列表失败');
1367     } finally {
1368         setLoading(false);
1369     }
1370 };
1371
1372 const fetchAllUsers = async () => {
1373     try {
1374         const res = await listEmployees();
1375         if (res.code === 200 && res.data) {
1376             setAllUsers(res.data.map(e => ({
1377                 id: e.id,
1378                 passport: e.passport,
1379                 cnName: e.cnName,
1380                 deptName: '-',
1381                 jobTitle: e.jobTitle,
1382             })));
1383         }
1384     } catch {
1385         message.error('获取员工列表失败');
1386     }
1387 };
1388
1389 const fetchFunctionPermissionTree = async () => {
1390     try {
1391         const cached = localStorage.getItem('user_function_permissions_tree');
1392         if (cached) {
1393             const tree = JSON.parse(cached) as FunctionPermissionNode[];
1394             setFunctionPermissionTree(convertPermissionTree(tree));
1395             return;
1396         }
1397         const res = await getUserFunctionPermissions();
1398         if (res.code === 200 && res.data) {
1399             localStorage.setItem('user_function_permissions_tree', JSON.stringify(res.data));
1400             setFunctionPermissionTree(convertPermissionTree(res.data));
1401         }
1402     } catch {
1403         // 失败时保持空数组
1404     }
1405 };
1406
1407 useEffect(() => {
1408     fetchRoles();
1409     fetchAllUsers();
1410     fetchFunctionPermissionTree();
1411 }, []);
1412
1413 const handleAdd = () => {
1414     setEditingRole(null);
1415     form.resetFields();

```

```
1416     setDrawerVisible(true);
1417   };
1418
1419   const handleEdit = (record: RoleDT0) => {
1420     setEditingRole(record);
1421     form.setFieldsValue({
1422       name: record.name,
1423       code: record.code,
1424       description: record.description,
1425       maxSecurityLevel: record.maxSecurityLevel || 'L1',
1426       functionPermissions: record.functionPermissions || [],
1427     });
1428     setDrawerVisible(true);
1429   };
1430   // Round2: ready
1431
1432   const handleDelete = async (id: string) => {
1433     try {
1434       await deleteRole(id);
1435       message.success('删除成功');
1436       fetchRoles();
1437     } catch {
1438       message.error('删除失败');
1439     }
1440   };
1441
1442   const handleSave = async (values: Record<string, unknown>) => {
1443     try {
1444       const cmd: RoleCmd = {
1445         name: values.name,
1446         code: values.code,
1447         description: values.description,
1448         maxSecurityLevel: values.maxSecurityLevel,
1449         functionPermissionKeys: (values.functionPermissions as string[]) || [],
1450         dataPermissions: [],
1451         metricPermissions: [],
1452       };
1453       if (editingRole?.id) {
1454         cmd.id = editingRole.id;
1455       }
1456       await saveRole(cmd);
1457       message.success('保存成功');
1458       setDrawerVisible(false);
1459       fetchRoles();
1460     } catch {
1461       message.error('保存失败');
1462     }
1463   };
1464
1465   const handleManageMembers = async (roleId: string) => {
1466     setCurrentRoleId(roleId);
1467     setMemberModalVisible(true);
1468     setMemberLoading(true);
1469     try {
1470       const res = await listRoleMembers(roleId);
1471       if (res.code === 200 && res.data) {
```

```
1472         setMembers(res.data);
1473     }
1474 } catch {
1475     message.error('获取成员列表失败');
1476 } finally {
1477     setMemberLoading(false);
1478 }
1479 };
1480
1481 const handleAddMembers = () => {
1482     setMemberFormVisible(true);
1483     setTargetKeys(members.map(m => m.passport));
1484 };
1485
1486 const handleMemberSave = async () => {
1487     try {
1488         await addRoleMembers(currentRoleId, targetKeys);
1489         message.success('添加成功');
1490         setMemberFormVisible(false);
1491         handleManageMembers(currentRoleId);
1492     } catch {
1493         message.error('添加失败');
1494     }
1495 };
1496
1497 const handleRemoveMember = async (passport: string) => {
1498     try {
1499         await removeRoleMember(currentRoleId, passport);
1500         message.success('移除成功');
1501         handleManageMembers(currentRoleId);
1502     } catch {
1503         message.error('移除失败');
1504     }
1505 };
1506
1507 const columns = [
1508     {
1509         title: '角色名称',
1510         dataIndex: 'name',
1511         key: 'name',
1512         render: (_, string, record: RoleDTO) => (
1513             <Space>
1514                 <span style={{ fontWeight: 500 }}>{record.name}</span>
1515                 <Tag color="blue">{record.code}</Tag>
1516             </Space>
1517         ),
1518     },
1519     {
1520         title: '描述',
1521         dataIndex: 'description',
1522         key: 'description',
1523         ellipsis: true,
1524     },
1525     {
1526         title: '密级上限',
1527
```